

Laporan Tugas Besar 3

TBFO (IF2224)



Disusun Oleh

Kelompok GooBurs:

Eliana Natalie Widjojo (13524116)

Muhammad Rafi Akbar (13524125)

Varistha Devi (13524135)

Ahmad Rinofaros Muchtar (13524138)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

BANDUNG 2025

Daftar Isi

Daftar Isi	2
Bab 1. Teori Singkat	3
1. Abstract Syntax Tree (AST)	3
2. Semantic Analysis	4
a. Symbol Table	5
b. Type Checking	6
c. Scope Resolution	7
d. Semantic Errors	7
e. Attribute Grammar dan Syntax Directed Translation	8
f. Decorated AST	8
3. Enumerated Type	9
Bab 2. Perancangan dan Implementasi	12
1. ast_builder	12
a. Fungsi Navigasi Parse Tree	12
b. Pembangunan Struktur Program	12
c. Pembangunan Tipe	12
d. Pembangunan Pernyataan dan Ekspresi	13
2. symbol_table	13
a. Struktur	13
b. Inisialisasi Predetermined	13
c. Scope management	14
3. semantic_analyzer	14
a. Entry Point dan Alur Umum	14
b. Fungsi Visit	14
c. Type System	16
d. Semantic Checking dan Error Handling	17
e. Penanganan Enumerated Type	17
f. Dekorasi Node AST	18
Bab 3. Pengujian	19
1. Kasus 1	19
2. Kasus 2	25
3. Kasus 3	37
4. Kasus 4	47
5. Kasus 5	56
6. Kasus 6	63
Bab 4. Kesimpulan dan Saran	70
Hasil Uji	70
Saran	70
Lampiran	70
Referensi	71

Bab 1. Teori Singkat

1. Abstract Syntax Tree (AST)

Abstract Syntax Tree (AST) merupakan representasi struktur program dalam bentuk tree (pohon) yang digunakan compiler setelah proses parsing. AST pada dasarnya adalah bentuk ringkas dari parse tree, dimana compiler hanya akan menyimpan informasi yang masih diperlukan untuk tahapan selanjutnya dan membuang informasi sintaksis yang sudah tidak lagi dibutuhkan.

Pada proses compiler, source code pertama kali akan melalui tahap lexical analysis untuk dipecah menjadi token-token. Token tersebut kemudian diproses parser pada tahap syntax analysis untuk membentuk parse tree sesuai aturan grammar bahasa pemrograman. Setelah parse tree terbentuk, compiler akan mengubahnya menjadi Abstract Syntax Tree (AST) yang lebih sederhana dan lebih mudah digunakan untuk semantic analysis maupun tahapan compiler selanjutnya.

Struktur dasar parse tree masih sangat bergantung pada aturan grammar yang digunakan parser untuk melakukan pengecekan sintaks. Oleh karena itu, parse tree biasanya masih menyimpan banyak node sintaksis tambahan seperti <expression>, <term>, <factor>, tanda kurung, titik koma, dan simbol grammar lainnya. Pada tahap semantic analysis, informasi tersebut sudah tidak terlalu diperlukan karena fokus compiler telah berpindah dari aturan sintaks menuju hubungan semantik antar operator dan operand.

AST dibentuk dengan cara menyederhanakan parse tree dan hanya mempertahankan informasi yang penting secara semantik. Sebagai contoh, ekspresi:

$2 + (z - 1)$

dapat direpresentasikan dalam bentuk AST sebagai:

$$\begin{array}{c} (+) \\ / \quad \backslash \\ 2 \quad (-) \\ \quad / \quad \backslash \\ \quad z \quad 1 \end{array}$$

Pada representasi tersebut, tanda kurung tidak lagi disimpan secara eksplisit karena hubungan parent-child pada tree sudah cukup untuk menunjukkan prioritas operasi. Node + dan - merepresentasikan operator, sedangkan 2, 1, dan z merepresentasikan operand berupa literal maupun variabel.

Secara umum, AST direpresentasikan sebagai struktur tree yang terdiri atas node-node dengan child tertentu. Setiap node menyimpan informasi semantik mengenai suatu konstruksi program, seperti assignment, operasi aritmatika, deklarasi variabel, conditional statement, procedure/function call, dan lain sebagainya. Sebagai contoh, node operasi biner (BinOpNode) memiliki child berupa operand kiri dan operand kanan, sedangkan node assignment (AssignNode) memiliki child berupa target variabel dan nilai yang akan diberikan.

Selain digunakan untuk merepresentasikan struktur program, AST juga memiliki peranan penting dalam semantic analysis dan static code analysis. Compiler dapat melakukan traversal terhadap AST untuk memeriksa berbagai aturan semantik, seperti:

- memastikan variabel telah dideklarasikan sebelum digunakan
- melakukan type checking
- melakukan scope checking
- membangun symbol table
- mendeteksi semantic error pada program

Pada proyek compiler Arion ini, AST dibangun dari parse tree menggunakan kelas ASTBuilder. Proses ini dilakukan dengan membuang node sintaksis yang tidak diperlukan dan mengubahnya menjadi node semantik yang lebih sederhana seperti AssignNode, BinOpNode, VarNode, IfNode, WhileNode, dan ProcedureCallNode. Dengan pendekatan tersebut, struktur program menjadi lebih ringkas dan lebih mudah dianalisis pada tahap semantic analysis.

2. Semantic Analysis

Semantic Analysis merupakan tahap ketiga dalam proses compiler setelah lexical analysis dan syntax analysis. Pada tahap ini, compiler tidak lagi hanya memeriksa apakah program sesuai dengan grammar bahasa pemrograman, tetapi juga memeriksa apakah program tersebut memiliki makna (semantically correct) sesuai aturan bahasa yang digunakan.

Jika syntax analysis berfokus pada struktur program, maka semantic analysis berfokus pada hubungan logis antar komponen program, seperti tipe data, deklarasi variabel, scope identifier, serta validitas operasi yang dilakukan pada program. Oleh karena itu, sebuah program dapat saja lolos tahap lexical dan syntax analysis tetapi tetap menghasilkan semantic error.

Sebagai contoh:

```
int a;  
a = "hello";
```

Secara sintaks, kode tersebut benar karena mengikuti aturan grammar bahasa pemrograman. Namun secara semantik, operasi assignment tersebut salah karena variabel bertipe integer diisi dengan string.

Semantic analysis bekerja menggunakan parse tree maupun Abstract Syntax Tree (AST) yang dihasilkan dari tahap parsing sebelumnya. Pada tahap ini, compiler akan melakukan traversal terhadap AST dan menambahkan informasi tambahan seperti tipe data, lexical scope, maupun referensi symbol table ke dalam node-node AST. Hasil akhir proses tersebut disebut sebagai decorated AST atau annotated syntax tree.

Secara umum, semantic analyzer memiliki beberapa fungsi utama, yaitu:

- melakukan type checking
- melakukan scope resolution
- memeriksa deklarasi identifier

- memvalidasi pemanggilan procedure/function
- membangun dan mengelola symbol table
- mendukung pembentukan intermediate representation pada tahap selanjutnya

a. Symbol Table

Symbol table merupakan struktur data penting yang digunakan compiler untuk menyimpan informasi mengenai identifier yang terdapat pada program, seperti nama variabel, fungsi, konstanta, parameter, maupun tipe data. Symbol table digunakan hampir pada seluruh tahapan compiler, baik pada proses analisis maupun sintesis program.

Setiap entri pada symbol table biasanya menyimpan informasi seperti:

- nama identifier
- tipe data
- scope
- kategori object
- memory location
- parameter function
- attribute tambahan lainnya

Sebagai contoh, deklarasi:

```
static int interest;
```

dapat direpresentasikan pada symbol table sebagai:

```
<interest, int, static>
```

Symbol table memiliki berbagai fungsi penting dalam compiler, antara lain:

- menyimpan seluruh identifier program
- memeriksa apakah variabel telah dideklarasikan
- melakukan type checking
- menentukan scope identifier
- membantu semantic analysis
- membantu code generation
- membantu proses optimisasi program

Pada semantic analysis, symbol table digunakan untuk:

- melakukan lookup identifier
- memeriksa deklarasi sebelum penggunaan
- memeriksa tipe data
- memvalidasi parameter procedure/function
- melakukan scope resolution

Compiler biasanya menyediakan operasi dasar pada symbol table, seperti:

- insert() untuk menambahkan identifier baru
- lookup() untuk mencari identifier
- set_attribute() untuk menambahkan atribut
- get_attribute() untuk mengambil informasi identifier

Dalam implementasinya, symbol table dapat dibuat menggunakan berbagai struktur data seperti:

- linear list
- linked list
- binary search tree
- hash table

Pada proyek compiler Arion, symbol table diimplementasikan menggunakan tiga tabel utama, yaitu:

- TAB untuk menyimpan identifier utama
- BTAB untuk menyimpan informasi block/scope
- ATAB untuk menyimpan metadata array

Selain itu, compiler Arion juga menggunakan mekanisme lexical level dan hierarchical scope untuk mendukung nested scope pada procedure maupun function.

b. Type Checking

Type checking merupakan proses pemeriksaan tipe data untuk memastikan bahwa variabel, expression, dan operator digunakan secara kompatibel sesuai aturan bahasa pemrograman. Type checking membantu compiler mendeteksi semantic error sebelum program dijalankan.

Sebagai contoh:

```
int x = "abc";
```

akan menghasilkan type mismatch karena integer tidak kompatibel dengan string.

Compiler melakukan type checking untuk:

- assignment statement
- expression arithmetic
- relational operation
- logical operation
- parameter procedure/function
- maupun return value function

Pada static type checking, proses pemeriksaan dilakukan saat compile time sehingga error dapat dideteksi lebih awal sebelum program dieksekusi. Bahasa seperti Pascal, C, dan Java menggunakan static type checking.

Selain itu, semantic analyzer juga dapat melakukan implicit type conversion (coercion). Sebagai contoh:

```
float x = 10.5;  
float y = x * 2;
```

nilai integer 2 dapat dikonversi secara otomatis menjadi 2.0.

Pada proyek compiler Arion, semantic analyzer melakukan pengecekan:

- assignment compatibility
- relational compatibility
- arithmetic compatibility

- conditional expression checking
- serta enum compatibility

c. Scope Resolution

Scope resolution merupakan proses menentukan dimana sebuah identifier dapat diakses pada program. Semantic analyzer harus memastikan bahwa identifier hanya digunakan pada scope yang valid.

Sebagai contoh:

```
{
  int x = 10;
}
x = 20;
```

akan menghasilkan semantic error karena variabel x sudah berada di luar scope ketika digunakan.

Compiler biasanya menggunakan hierarchical symbol table untuk menangani nested scope. Scope lokal hanya dapat diakses pada block tempat identifier tersebut dideklarasikan, sedangkan scope global dapat diakses oleh seluruh bagian program.

Pada compiler Arion, scope management dilakukan menggunakan:

- pushScope()
- popScope()
- lexical level (lev)
- mekanisme display

Dengan pendekatan tersebut, semantic analyzer dapat melakukan pencarian identifier mulai dari scope terdalam hingga global scope.

d. Semantic Errors

Semantic error merupakan error yang muncul ketika program benar secara sintaks, tetapi salah secara makna atau aturan bahasa pemrograman.

Beberapa semantic error yang umum terjadi antara lain:

- type mismatch
- undeclared variable
- multiple declaration
- reserved keyword misuse
- actual dan formal parameter mismatch
- penggunaan variabel di luar scope
- invalid operation pada tipe data tertentu

Sebagai contoh:

```
x = 10;
```

akan menghasilkan semantic error apabila variabel x belum pernah dideklarasikan sebelumnya.

Pada compiler Arion, semantic error dideteksi selama traversal AST dan akan menghasilkan pesan error apabila aturan semantik program dilanggar.

e. Attribute Grammar dan Syntax Directed Translation

Semantic analysis umumnya menggunakan konsep attribute grammar dan syntax directed translation (SDT). Pada metode ini, aturan grammar tidak hanya digunakan untuk memeriksa sintaks, tetapi juga diberikan semantic rules untuk menentukan makna dari produksi grammar tersebut.

Sebagai contoh:

$$E \rightarrow E + T$$

secara grammar hanya menunjukkan bentuk expression penjumlahan. Agar compiler dapat memahami maknanya, diperlukan semantic rule tambahan seperti:

$$E.value = E.value + T.value$$

Aturan tersebut menunjukkan bahwa nilai expression E diperoleh dari hasil penjumlahan nilai child node E dan T.

Attribute grammar menggunakan atribut untuk menyimpan informasi semantik pada parse tree maupun AST. Secara umum atribut dibagi menjadi dua jenis:

- synthesized attributes
- inherited attributes

Synthesized attribute memperoleh nilai dari child node, sedangkan inherited attribute memperoleh nilai dari parent maupun sibling node.

Dalam semantic analysis modern, semantic rule tersebut biasanya diterapkan melalui traversal terhadap AST sehingga compiler dapat melakukan evaluasi tipe, pengecekan scope, dan anotasi node secara sistematis.

f. Decorated AST

Decorated AST atau Annotated AST merupakan AST yang telah diperkaya dengan informasi semantik tambahan hasil proses semantic analysis. Informasi tambahan tersebut dapat berupa:

- tipe data
- symbol table reference
- lexical level
- scope
- attribute semantic lainnya

Sebagai contoh:

$$a + b$$

setelah semantic analysis dapat berubah menjadi:

$$\begin{array}{l} a : \text{integer} \\ b : \text{integer} \end{array}$$

$(a + b) : \text{integer}$

Dengan adanya anotasi tersebut, compiler dapat memahami konteks semantic setiap node pada AST.

Pada spesifikasi tugas compiler Arion, semantic analysis melakukan anotasi terhadap AST menggunakan tiga symbol table utama, yaitu:

- TAB
- BTAB
- ATAB

Decorated AST yang dihasilkan akan digunakan untuk proses semantic checking dan tahapan compiler selanjutnya. Selain itu, setiap node AST juga dapat diberikan informasi tambahan seperti:

- TypeCode
- tabIndex
- lexical level
- referensi scope aktif

Dengan pendekatan tersebut, compiler dapat melakukan semantic checking secara lebih sistematis dan terstruktur.

Pada implementasi compiler Arion, node AST yang berkaitan dengan enumerated type juga akan dianotasi menggunakan TYPE_ENUM. Dengan demikian, semantic analyzer dapat membedakan identifier enum dari integer biasa serta melakukan semantic checking yang lebih aman pada assignment maupun expression relasional.

3. Enumerated Type

Enumerated type atau enum merupakan tipe data yang berisi sekumpulan nilai konstan yang telah ditentukan sebelumnya. Enumerated type pertama kali diperkenalkan pada bahasa Pascal untuk memungkinkan programmer menggunakan nama yang lebih deskriptif dan mudah dipahami dibandingkan dengan menggunakan angka secara langsung.

Secara umum, enumerated type digunakan untuk merepresentasikan sekumpulan nilai terbatas, seperti:

- hari dalam seminggu
- bulan
- warna
- status
- kategori tertentu

Sebagai contoh:

```
type
  Color = (red, green, blue);
```

Pada contoh tersebut, Color merupakan enumerated type yang memiliki tiga nilai valid, yaitu red, green, dan blue.

Enumerated type memungkinkan programmer menggunakan nama yang lebih self-documenting dibandingkan dengan menggunakan integer biasa. Dengan demikian, kode program menjadi lebih mudah dibaca dan dipahami.

Secara internal, compiler biasanya akan memetakan setiap elemen enum ke suatu nilai ordinal tertentu. Sebagai contoh:

```
red → 0  
green → 1  
blue → 2
```

Meskipun direpresentasikan menggunakan bilangan ordinal, setiap enumerated type tetap dianggap sebagai tipe data yang berbeda secara semantik. Oleh karena itu, operasi antar enum yang berbeda tidak diperbolehkan.

Sebagai contoh:

```
type  
  Color = (red, green, blue);  
  Month = (jan, feb, mar);
```

maka operasi:

```
red = green
```

masih valid karena keduanya berasal dari tipe Color, sedangkan:

```
red = jan
```

seharusnya menghasilkan semantic error karena berasal dari enumerated type yang berbeda.

Pada banyak bahasa pemrograman modern, enumerated type umumnya direpresentasikan menggunakan integer sebagai underlying representation karena integer lebih efisien diproses oleh mesin dan lebih mudah digunakan pada operasi komputasi. Namun demikian, compiler tetap mempertahankan informasi tipe enum agar semantic analyzer dapat membedakan antar-enumerated type yang berbeda.

Bahasa seperti C dan C++ secara umum menggunakan integer sebagai representasi internal enum, sedangkan bahasa seperti Java dan C# memperlakukan enum sebagai tipe yang lebih kuat (strongly typed enum). Pada pendekatan tersebut, enum tidak hanya menyimpan nilai ordinal, tetapi juga dapat memiliki method maupun behavior tambahan.

Dalam teori compiler, enumerated type biasanya diperlakukan sebagai ordinal type, yaitu tipe data yang memiliki urutan nilai tertentu. Oleh karena itu:

- elemen enum dapat dibandingkan
- memiliki ordering
- dapat digunakan sebagai indeks array

Sebagai contoh:

```
type  
  Day = (monday, tuesday, wednesday);
```

maka operasi:

```
monday < tuesday
```

masih memiliki makna yang valid karena setiap elemen enum memiliki urutan ordinal tertentu.

Pada proyek compiler Arion, enumerated type diimplementasikan sebagai tipe data khusus menggunakan TYPE_ENUM. Semantic analyzer akan:

- mendaftarkan identifier enum ke symbol table
- memberikan ordinal value untuk setiap elemen enum
- melakukan type checking terhadap enum
- memastikan bahwa operasi antar enum hanya dilakukan pada enum dengan tipe yang kompatibel.

Sebagai contoh:

```
type
  Color = (red, green, blue);

var
  c : Color;

begin
  c := red;
end.
```

Pada semantic analysis, compiler akan:

- mendaftarkan Color sebagai TYPE_ENUM
- mendaftarkan red, green, dan blue sebagai constant enum
- melakukan anotasi tipe enum pada AST maupun symbol table

Dengan pendekatan tersebut, semantic analyzer dapat membedakan enumerated type dari integer biasa sehingga proses type checking menjadi lebih aman dan lebih sesuai dengan konsep semantic type system pada compiler modern.

Bab 2. Perancangan dan Implementasi

1. `ast_builder`

ASTBuilder bertanggung jawab mengonversi parse tree hasil Parser menjadi Abstract Syntax Tree (AST) yang lebih ringkas. Konversi dilakukan dengan metode `build(ParseNode* root)` untuk entry point, yang kemudian mendelegasikan ke fungsi-fungsi `build*` sesuai label node.

Komponen utama:

a. Fungsi Navigasi Parse Tree

- `isLabel(node, label)` – mengecek apakah label non-terminal sesuai.
- `isTerminal(node)` – mengecek apakah node adalah terminal.
- `child(node, label)` / `children(node, label)` – mengambil child pertama atau semua child dengan label tertentu.
- `firstTerminal` / `allTerminals` – mencari node terminal berdasarkan nama token.
- `identifierList(node)` – mengekstrak semua nilai identifier dari suatu sub-tree.
- `operatorFromNode(node)` – memetakan nama token ke simbol operator (contoh: `rdiv` → `/`, `idiv` → `div`, `eql` → `=`).

b. Pembangunan Struktur Program

- `buildProgram` → `ProgramNode` berisi nama program, deklarasi, dan blok utama.
- `buildDeclarationPart` → mendistribusikan ke `buildConstDecl`, `buildTypeDecl`, `buildVarDecl`, `buildSubprogramDecl`.
- `buildVarDecl` → menghasilkan satu `VarDeclNode` per variabel dalam `<identifier-list>`, masing-masing menyimpan nama dan tipe.
- `buildProcedureDecl` / `buildFunctionDecl` → `ProcedureDeclNode` / `FunctionDeclNode` beserta parameter formal dan blok body-nya.

c. Pembangunan Tipe

- `buildType` → mendistribusikan ke `buildArrayType`, `buildRange`, `buildEnumerated`, `buildRecordType`, atau membuat node tipe dasar (`ident`).
- `buildArrayType` → `ArrayTypeNode` yang menyimpan index type dan element type.
- `buildRange` → `RangeTypeNode` dengan batas bawah (`low`) dan batas atas (`high`).
- `buildEnumerated` → `EnumTypeNode` berisi daftar identifier konstanta.

- buildRecordType → RecordTypeNode berisi field-field yang dihasilkan buildFieldList.

d. Pembangunan Pernyataan dan Ekspresi

- buildAssignment → AssignNode (target variabel dan ekspresi nilai).
- buildIf → IfNode dengan kondisi, cabang then, dan cabang else opsional.
- buildWhile → WhileNode; buildRepeat → RepeatNode; buildFor → ForNode dengan flag downto.
- buildProcedureFunctionCall → ProcedureCallNode beserta argumen.
- buildExpression → menangani operator relasional; buildSimpleExpression → operator aditif; buildTerm → operator multiplikatif; buildFactor → literal, variabel, NOT, ekspresi berkurung.
- buildVariable → VarNode dengan applyComponent untuk akses array ([...]) dan field record (.field).

2. symbol_table

SymbolTable menyimpan seluruh informasi deklarasi identifikasi menggunakan tiga tabel terpisah.

a. Struktur

- tab (vector TabEntry)
Menyimpan setiap identifier dengan atribut: identifier (nama), link (pointer ke identifier sebelumnya dalam blok yang sama), obj (kelas: konstanta/variabel/tipe/prosedur/fungsi), type (TypeCode), ref (indeks ke atab/btab), nrm (1=normal, 0=var parameter), lev (lexical level), adr (offset/nilai). Indeks 0–32 dicadangkan untuk reserved words.
- btab (vector BTabEntry)
Menyimpan informasi blok: last (indeks tab terakhir dalam blok), lpar (parameter terakhir), psze (ukuran parameter), vsze (ukuran variabel lokal).
- atab (vector ATabEntry)
Menyimpan deskriptor array: xtyp (tipe indeks), etyp (tipe elemen), eref (referensi elemen komposit), low/high (batas indeks), elsz (ukuran elemen), size (total ukuran = (high–low+1) × elsz).

b. Inisialisasi Predetermined

Konstruktor SymbolTable memanggil `initPredefined()` yang mengisi 32 reserved words (indeks 0–31), kemudian mendaftarkan predefined identifier mulai indeks 32:

Tipe dasar: integer, real, char, boolean, string (sebagai OBJ_TYPE).

Konstanta boolean: true (adr=1), false (adr=0).

Prosedur bawaan: `writeln`, `readln`.

c. Scope management

- `pushScope()` → menaikkan `currentLevel`, membuat BTabEntry baru di btab, dan memperbarui array `display` (di mana `display[level]` menyimpan indeks btab dari blok aktif pada level tersebut).
- `popScope()` → menurunkan `currentLevel`.
- `enter(name, obj, type, ...)` → mendaftarkan identifier baru di blok saat ini; memeriksa redeclaration dalam scope yang sama menggunakan linked list per blok; memperbarui `btab[currentBlock].last`.
- `lookup(name)` → penelusuran case-insensitive dari level terdalam ke level 0 menggunakan rantai link.
- `enterArray(...)` → mendaftarkan deskriptor array ke atab dan mengembalikan indeksnya.

3. semantic_analyzer

a. Entry Point dan Alur Umum

SemanticAnalyzer melakukan dua pekerjaan utama sekaligus dalam satu penelusuran parse tree: (1) membangun Decorated AST, dan (2) menjalankan pemeriksaan semantik.

Metode `analyze(ParseNode* root)` memanggil `visitProgram` sebagai titik masuk. Seluruh kunjungan berjalan secara Depth-First Search (DFS) mengikuti pola L-Attributed Grammar: atribut dihitung dari kiri ke kanan dan dari bawah ke atas (synthesized attributes), kemudian digunakan oleh node induk untuk pemeriksaan dan anotasi.

b. Fungsi Visit

- `visitProgram`
Mendaftarkan nama program, memproses deklarasi, dan mengunjungi blok utama
- `visitDeclarationPart`
Mendistribusikan ke `visitConstDecl`, `visitTypeDecl`, `visitVarDecl`, `visitSubprogramDecl`

- visitConstDecl
Mendaftarkan konstanta ke tab dengan tipe yang diinferensi dari nilai literal
- visitTypeDecl
Mendaftarkan alias tipe ke symbol table. Untuk array, semantic analyzer juga mendaftarkan metadata array ke ATAB melalui registerArrayType. Untuk enumerated type, semantic analyzer mendaftarkan nama tipe enum sebagai OBJ_TYPE dengan TYPE_ENUM, kemudian setiap elemen enum didaftarkan sebagai OBJ_CONSTANT dengan TYPE_ENUM dan diberikan ordinal value pada atribut adr.
- visitVarDecl
Mendaftarkan variabel ke tab dengan TypeCode yang diselesaikan dari nama tipe
- visitProcDecl / visitFuncDecl
Mendaftarkan prosedur/fungsi ke tab, memanggil pushScope, memproses parameter formal dan blok body, lalu popScope
- visitBlock / visitCompoundStmt
Mengunjungi statement list
- visitAssignStmt
Lookup variabel target, kunjungi ekspresi, periksa isAssignCompatible
- visitIfStmt
Kunjungi ekspresi kondisi, periksa tipe Boolean via checkConditionType
- visitWhileStmt / visitRepeatStmt
Serupa dengan if; kondisi harus bertipe Boolean
- visitForStmt
Periksa bahwa variabel iterator bertipe Integer; batas harus kompatibel
- visitProcCall
Lookup nama prosedur/fungsi di tab, kunjungi argumen
- visitExpression
Menangani operator relasional; hasil selalu TYPE_BOOLEAN

- visitSimpleExpr
Operator aditif (+, -, or); tipe hasil dihitung via resultTypeOfBinOp
- visitTerm
Operator multiplikatif (*, /, div, mod, and)
- visitFactor
Literal (int/real/char/string), variabel, NOT, ekspresi berkurung
- visitVariable
Lookup identifier di tab via lookupSymbol; dekorasi node dengan tab_index, type, lev

c. Type System

TypeCode terdiri dari: TYPE_INTEGER, TYPE_REAL, TYPE_CHAR, TYPE_BOOLEAN, TYPE_STRING, TYPE_ARRAY, TYPE_RECORD, TYPE_SUBRANGE, TYPE_ENUM, TYPE_NONE

Aturan kompatibilitas tipe yang diimplementasikan:

- isCompatible(t1, t2) → dua tipe kompatibel jika sama, atau salah satu adalah subrange dari yang lain, atau keduanya TYPE_STRING.
- isAssignCompatible(target, value) → selain kompatibel, juga mengizinkan value=INTEGER di-assign ke target=REAL (widening).
- isRelationalCompatible(t1, t2) → untuk operator relasional, mengizinkan perbandingan INTEGER dengan REAL.
- resultTypeOfBinOp(op, left, right) → operator / selalu menghasilkan REAL; div/mod menghasilkan INTEGER; and/or/not menghasilkan BOOLEAN; aritmetika INTEGER op INTEGER → INTEGER, jika salah satu REAL → REAL.

Untuk enumerated type, semantic analyzer memperlakukan enum sebagai ordinal type yang tetap memiliki semantic type tersendiri. Setiap elemen enum memiliki ordinal value internal, tetapi compiler tetap mempertahankan TYPE_ENUM agar enum tidak langsung diperlakukan sebagai integer biasa.

Sebagai contoh:

```
type
  Color = (red, green, blue);
```

maka:

red → ordinal 0

green → ordinal 1

blue → ordinal 2

Semantic analyzer memperbolehkan assignment maupun comparison antar identifier enum yang kompatibel, seperti:

```
c := red  
c = green
```

Namun operasi antara enum berbeda tipe dapat dianggap tidak kompatibel secara semantik.

d. Semantic Checking dan Error Handling

- `lookupSymbol(name, line)` → mencari identifier; jika tidak ditemukan, memanggil `semanticError` dan melanjutkan (tidak crash).
- `checkAssignment(target, value, context, line)` → melempar pesan error jika tidak `isAssignCompatible`.
- `checkConditionType(type, context, line)` → error jika tipe bukan `TYPE_BOOLEAN`.
- `semanticError(msg, line)` → menambahkan pesan ke `std::vector<std::string> errors`.
- Setelah `analyze` selesai, `main.cpp` memeriksa `semanalyzer.hasErrors()` dan mencetak seluruh error via `printErrors`; return code non-zero jika ada error.

e. Penanganan Enumerated Type

Enumerated type diimplementasikan menggunakan `TYPE_ENUM` sebagai semantic type khusus. Ketika semantic analyzer menemukan deklarasi enum, compiler akan mendaftarkan nama tipe enum sebagai `OBJ_TYPE` ke symbol table. Setelah itu, setiap elemen enum akan dimasukkan sebagai `OBJ_CONSTANT` dengan `TYPE_ENUM`.

Nilai ordinal enum disimpan pada atribut `adr` di `TAB`. Sebagai contoh:

```
type  
Color = (red, green, blue);
```

akan menghasilkan:

red → `adr = 0`

green → `adr = 1`

blue → `adr = 2`

Pendekatan ini memungkinkan semantic analyzer melakukan pengecekan semantic terhadap enum secara lebih aman dibandingkan jika enum langsung diperlakukan sebagai integer biasa.

Selain itu, Decorated AST juga akan menyimpan informasi TYPE_ENUM pada node enum sehingga semantic analyzer dapat melakukan pengecekan assignment, relational expression, maupun penggunaan enum pada statement seperti case statement.

f. Dekorasi Node AST

Setiap node AST yang dihasilkan oleh visit* dianotasi dengan:

- type → TypeCode dari ekspresi/identifier tersebut.
- tab_index → indeks entri di tab (untuk variabel, konstanta, dan prosedur/fungsi).
- lev → lexical level tempat identifier dideklarasikan.

Bab 3. Pengujian

1. Kasus 1

Input:

```
program BasicTypes;  
var  
  i: integer;  
  r: real;  
  c: char;  
  b: boolean;  
begin  
  i := 42;  
  r := 3.14;  
  c := 'A';  
  b := true;  
  r := i;  
end.
```

Output

```
programsy  
ident (BasicTypes)  
semicolon  
varsy  
ident (i)  
colon  
ident (integer)  
semicolon  
ident (r)  
colon  
ident (real)  
semicolon  
ident (c)  
colon  
ident (char)  
semicolon  
ident (b)  
colon  
ident (boolean)  
semicolon  
beginsy  
ident (i)  
becomes  
intcon (42)  
semicolon
```

```

ident (r)
becomes
realcon (3.14)
semicolon
ident (c)
becomes
charcon ('A')
semicolon
ident (b)
becomes
ident (true)
semicolon
ident (r)
becomes
ident (i)
semicolon

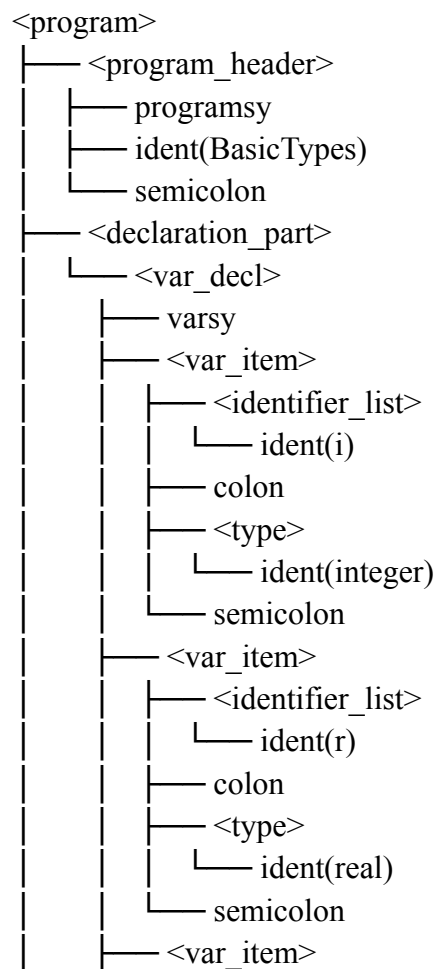
```

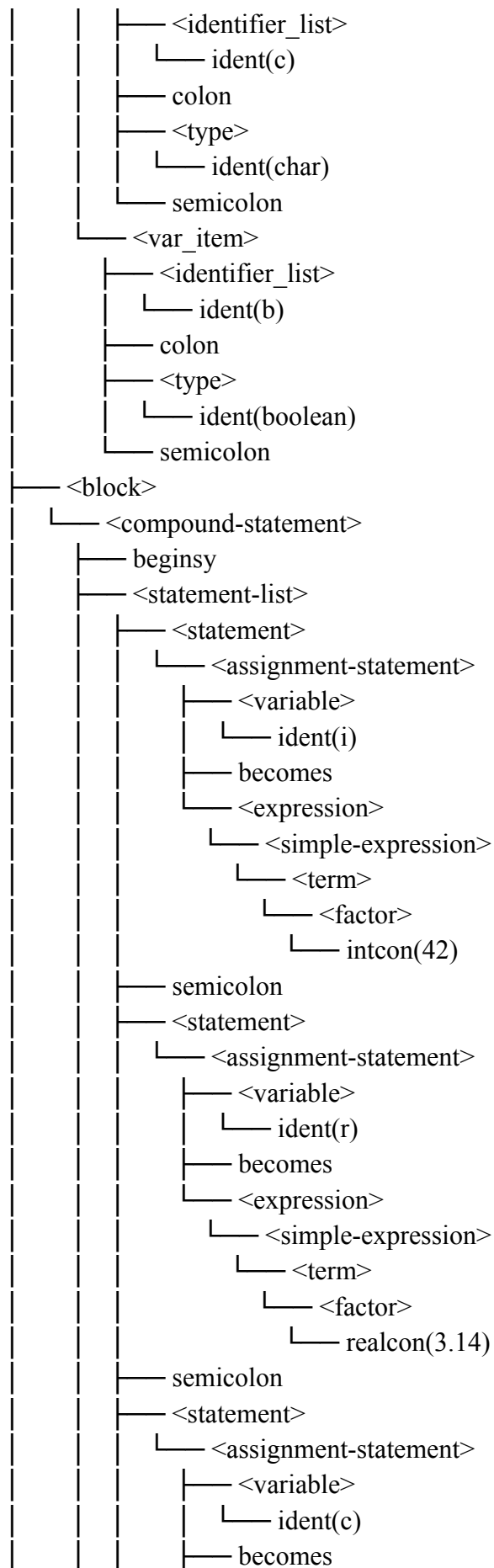
```

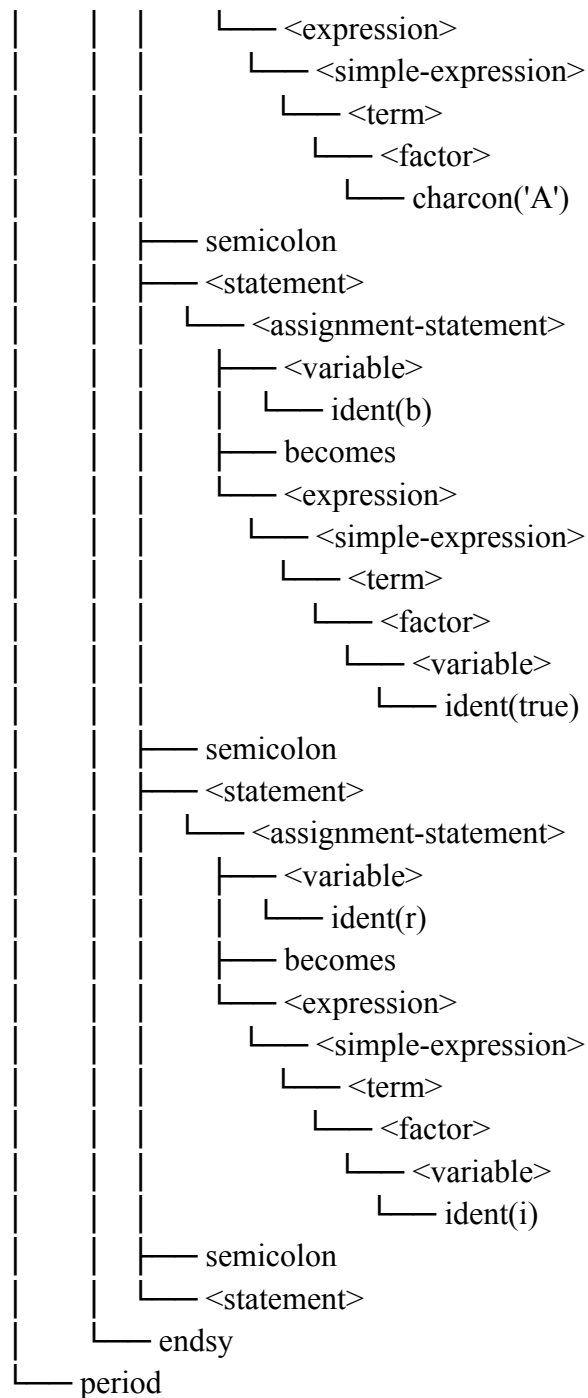
endsy
period

```

--- PARSE TREE ---







--- ABSTRACT SYNTAX TREE ---

ProgramNode(name: 'BasicTypes')

Declarations

VarDecl(name: 'i')

TypeName(integer)

VarDecl(name: 'r')

TypeName(real)

VarDecl(name: 'c')

TypeName(char)

VarDecl(name: 'b')

```
    TypeName(boolean)
Block
  BlockNode
    Assign
      target:
        Var('i')
      value:
        IntegerNumber(42)
    Assign
      target:
        Var('r')
      value:
        RealNumber(3.14)
    Assign
      target:
        Var('c')
      value:
        Char('A')
    Assign
      target:
        Var('b')
      value:
        Var('true')
    Assign
      target:
        Var('r')
      value:
        Var('i')
    Empty
```

--- DECORATED AST ---

```
ProgramNode(name: 'BasicTypes')
  Declarations
    VarDecl(name: 'i')
      TypeName(integer)
    VarDecl(name: 'r')
      TypeName(real)
    VarDecl(name: 'c')
      TypeName(char)
    VarDecl(name: 'b')
      TypeName(boolean)
  Block
    BlockNode
      Assign
```

```

target:
  Var('i' : integer [tab=43, lev=0])
value:
  IntegerNumber(42 : integer)
Assign
target:
  Var('r' : real [tab=44, lev=0])
value:
  RealNumber(3.14 : real)
Assign
target:
  Var('c' : char [tab=45, lev=0])
value:
  Char('A' : char)
Assign
target:
  Var('b' : boolean [tab=46, lev=0])
value:
  Var('true' : boolean [tab=38, lev=0])
Assign
target:
  Var('r' : real [tab=44, lev=0])
value:
  Var('i' : integer [tab=43, lev=0])
Empty

```

===== TAB =====

IDX	IDENT	LINK	OBJ	TYPE	REF	NRM	LEV
ADR							
0	and	0	2	0	0	1	0
1	array	0	2	0	0	1	0
2	begin	0	2	0	0	1	0
3	case	0	2	0	0	1	0
4	const	0	2	0	0	1	0
5	div	0	2	0	0	1	0
6	downto	0	2	0	0	1	0
7	do	0	2	0	0	1	0
8	else	0	2	0	0	1	0
9	end	0	2	0	0	1	0
10	for	0	2	0	0	1	0
11	function	0	2	0	0	1	0
12	if	0	2	0	0	1	0
13	mod	0	2	0	0	1	0
14	not	0	2	0	0	1	0

15	of	0	2	0	0	1	0	0
16	or	0	2	0	0	1	0	0
17	procedure	0	2	0	0	1	0	0
18	program	0	2	0	0	1	0	0
19	record	0	2	0	0	1	0	0
20	repeat	0	2	0	0	1	0	0
21	integer	0	2	0	0	1	0	0
22	real	0	2	0	0	1	0	0
23	boolean	0	2	0	0	1	0	0
24	char	0	2	0	0	1	0	0
25	string	0	2	0	0	1	0	0
26	then	0	2	0	0	1	0	0
27	to	0	2	0	0	1	0	0
28	type	0	2	0	0	1	0	0
29	until	0	2	0	0	1	0	0
30	var	0	2	0	0	1	0	0
31	while	0	2	0	0	1	0	0
32	(reserved)	0	2	0	0	1	0	0
33	integer	0	2	1	0	1	0	0
34	real	33	2	2	0	1	0	0
35	char	34	2	3	0	1	0	0
36	boolean	35	2	4	0	1	0	0
37	string	36	2	5	0	1	0	0
38	true	37	0	4	0	1	0	1
39	false	38	0	4	0	1	0	0
40	writeln	39	3	0	0	1	0	0
41	readln	40	3	0	0	1	0	0
42	basictypes	41	5	0	0	1	0	0
43	i	42	1	1	0	1	0	0
44	r	43	1	2	0	1	0	0
45	c	44	1	3	0	1	0	0
46	b	45	1	4	0	1	0	0

===== BTAB =====

IDX	LAST	LPAR	PSZE	VSZE
0	46	0	0	4

===== ATAB =====

IDX	XTYP	ETYP	EREF	LOW	HIGH	ELSZ	SIZE
-----	------	------	------	-----	------	------	------

2. Kasus 2

Input:

```

program Arithmetic;
var
  a, b: integer;
  x, y: real;
  res: real;
begin
  a := 10;
  b := 3;
  x := 2.5;
  y := 1.5;
  res := a + b;
  res := x * y;
  res := a / b;
  a := a div b;
  a := a mod b;
  res := x - y;
end.

```

Output

```

programsy
ident (Arithmetic)
semicolon
varsy
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon
ident (x)
comma
ident (y)
colon
ident (real)
semicolon
ident (res)
colon
ident (real)
semicolon
beginsy
ident (a)
becomes
intcon (10)
semicolon

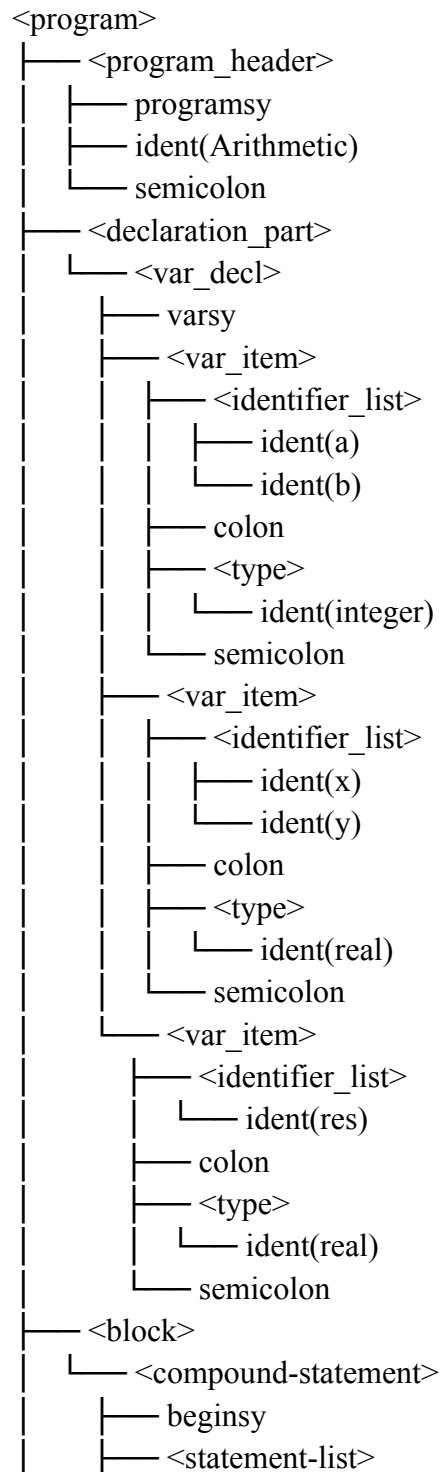
```

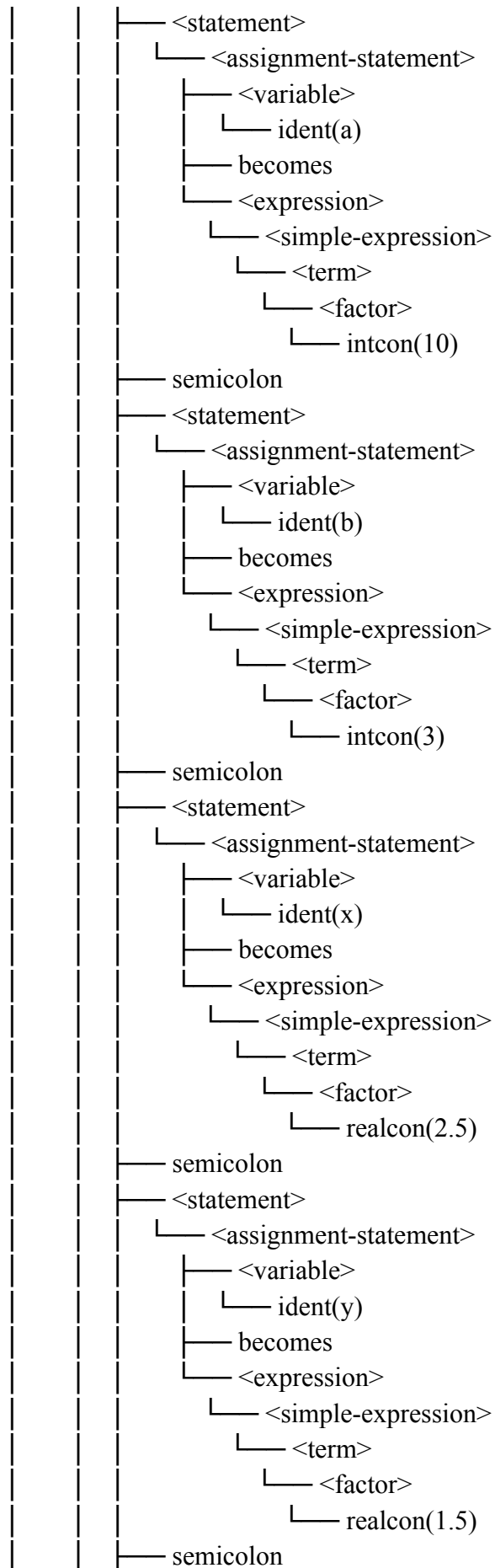
ident (b)
becomes
intcon (3)
semicolon
ident (x)
becomes
realcon (2.5)
semicolon
ident (y)
becomes
realcon (1.5)
semicolon
ident (res)
becomes
ident (a)
plus
ident (b)
semicolon
ident (res)
becomes
ident (x)
times
ident (y)
semicolon
ident (res)
becomes
ident (a)
rdiv
ident (b)
semicolon
ident (a)
becomes
ident (a)
idiv
ident (b)
semicolon
ident (a)
becomes
ident (a)
imod
ident (b)
semicolon
ident (res)
becomes

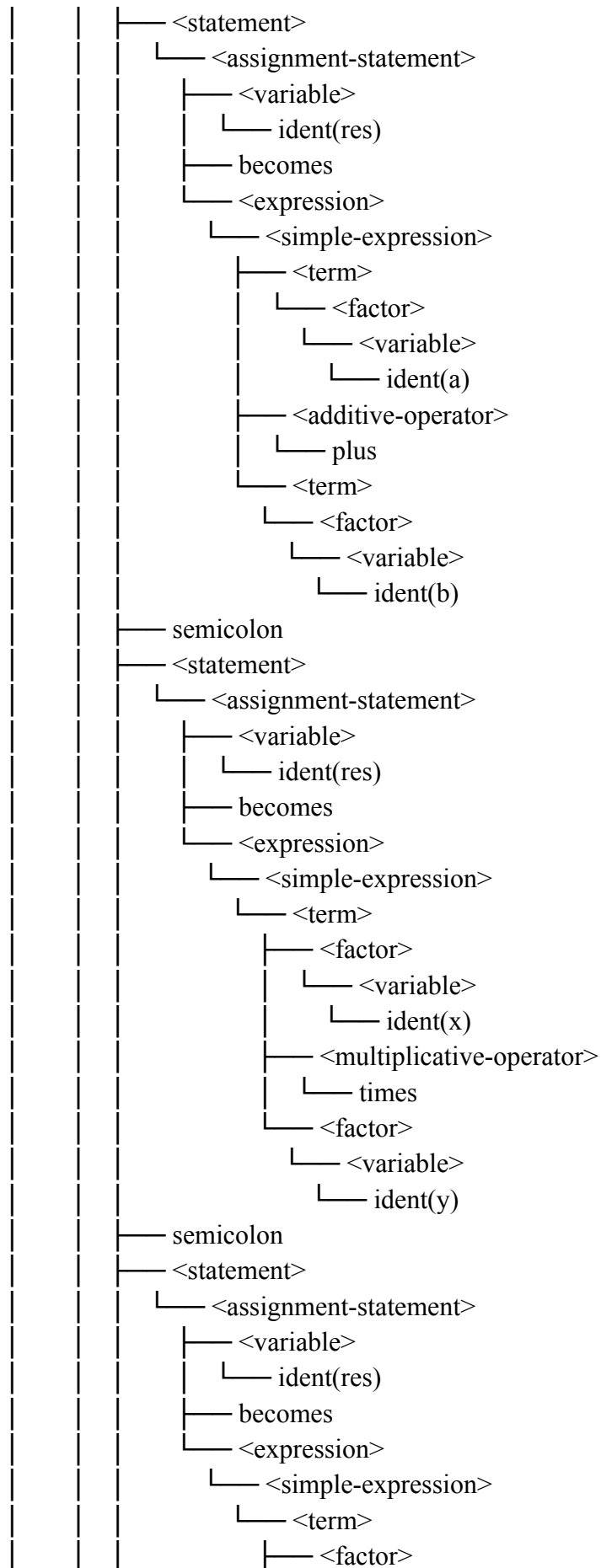
ident (x)
minus
ident (y)
semicolon

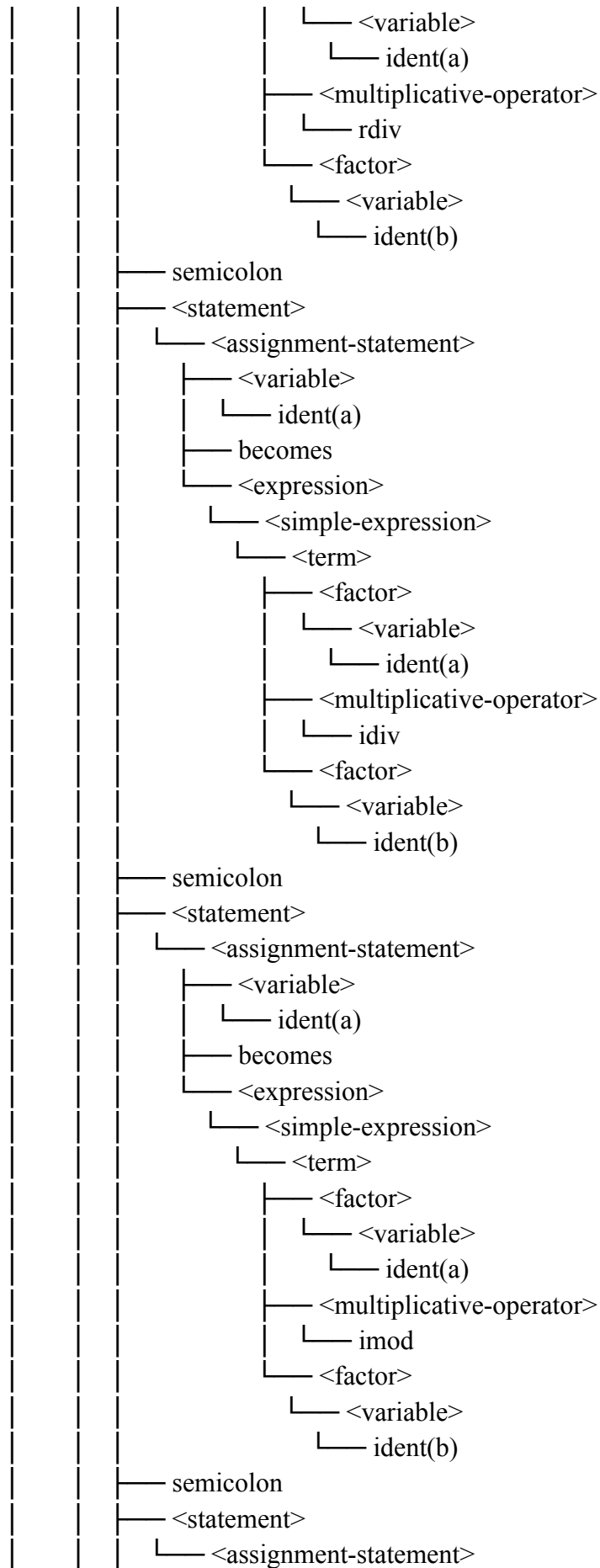
endsy
period

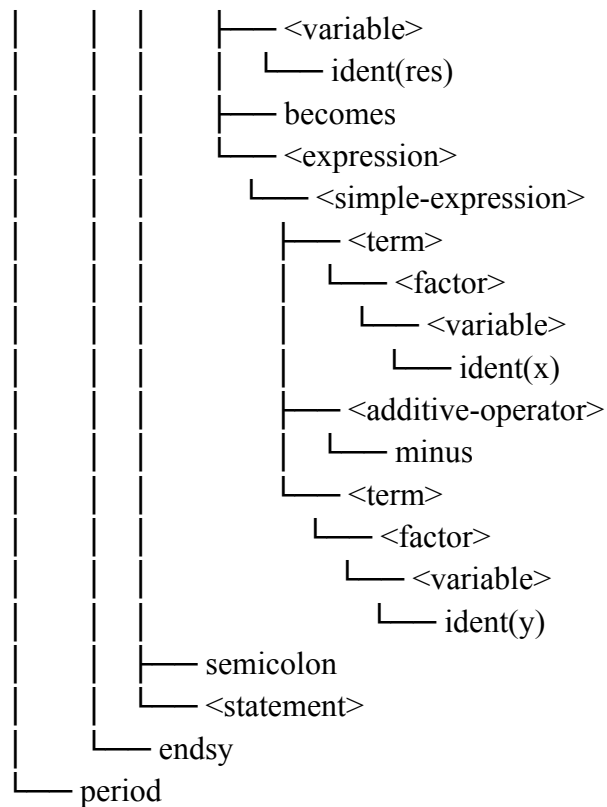
--- PARSE TREE ---











--- ABSTRACT SYNTAX TREE ---

ProgramNode(name: 'Arithmetic')

Declarations

VarDecl(name: 'a')

TypeName(integer)

VarDecl(name: 'b')

TypeName(integer)

VarDecl(name: 'x')

TypeName(real)

VarDecl(name: 'y')

TypeName(real)

VarDecl(name: 'res')

TypeName(real)

Block

BlockNode

Assign

target:

Var('a')

value:

IntegerNumber(10)

Assign

target:

Var('b')

value:


```
    IntegerNumber(3)
Assign
  target:
    Var('x')
  value:
    RealNumber(2.5)
Assign
  target:
    Var('y')
  value:
    RealNumber(1.5)
Assign
  target:
    Var('res')
  value:
    BinOp('+')
      Var('a')
      Var('b')
Assign
  target:
    Var('res')
  value:
    BinOp('*')
      Var('x')
      Var('y')
Assign
  target:
    Var('res')
  value:
    BinOp('/')
      Var('a')
      Var('b')
Assign
  target:
    Var('a')
  value:
    BinOp('div')
      Var('a')
      Var('b')
Assign
  target:
    Var('a')
  value:
    BinOp('mod')
```

```
    Var('a')
    Var('b')
Assign
  target:
    Var('res')
  value:
    BinOp('-',
      Var('x')
      Var('y')
    )
Empty
```

--- DECORATED AST ---

```
ProgramNode(name: 'Arithmetic')
Declarations
  VarDecl(name: 'a')
    TypeName(integer)
  VarDecl(name: 'b')
    TypeName(integer)
  VarDecl(name: 'x')
    TypeName(real)
  VarDecl(name: 'y')
    TypeName(real)
  VarDecl(name: 'res')
    TypeName(real)
Block
  BlockNode
    Assign
      target:
        Var('a' : integer [tab=43, lev=0])
      value:
        IntegerNumber(10 : integer)
    Assign
      target:
        Var('b' : integer [tab=44, lev=0])
      value:
        IntegerNumber(3 : integer)
    Assign
      target:
        Var('x' : real [tab=45, lev=0])
      value:
        RealNumber(2.5 : real)
    Assign
      target:
        Var('y' : real [tab=46, lev=0])
```

```

value:
  RealNumber(1.5 : real)
Assign
target:
  Var('res' : real [tab=47, lev=0])
value:
  BinOp('+ : integer)
    Var('a' : integer [tab=43, lev=0])
    Var('b' : integer [tab=44, lev=0])
Assign
target:
  Var('res' : real [tab=47, lev=0])
value:
  BinOp('* : real)
    Var('x' : real [tab=45, lev=0])
    Var('y' : real [tab=46, lev=0])
Assign
target:
  Var('res' : real [tab=47, lev=0])
value:
  BinOp('/ : real)
    Var('a' : integer [tab=43, lev=0])
    Var('b' : integer [tab=44, lev=0])
Assign
target:
  Var('a' : integer [tab=43, lev=0])
value:
  BinOp('div' : integer)
    Var('a' : integer [tab=43, lev=0])
    Var('b' : integer [tab=44, lev=0])
Assign
target:
  Var('a' : integer [tab=43, lev=0])
value:
  BinOp('mod' : integer)
    Var('a' : integer [tab=43, lev=0])
    Var('b' : integer [tab=44, lev=0])
Assign
target:
  Var('res' : real [tab=47, lev=0])
value:
  BinOp('- : real)
    Var('x' : real [tab=45, lev=0])
    Var('y' : real [tab=46, lev=0])

```

Empty

===== TAB =====

IDX	IDENT	LINK	OBJ	TYPE	REF	NRM	LEV
ADR							
0	and	0	2	0	0	1	0
1	array	0	2	0	0	1	0
2	begin	0	2	0	0	1	0
3	case	0	2	0	0	1	0
4	const	0	2	0	0	1	0
5	div	0	2	0	0	1	0
6	downto	0	2	0	0	1	0
7	do	0	2	0	0	1	0
8	else	0	2	0	0	1	0
9	end	0	2	0	0	1	0
10	for	0	2	0	0	1	0
11	function	0	2	0	0	1	0
12	if	0	2	0	0	1	0
13	mod	0	2	0	0	1	0
14	not	0	2	0	0	1	0
15	of	0	2	0	0	1	0
16	or	0	2	0	0	1	0
17	procedure	0	2	0	0	1	0
18	program	0	2	0	0	1	0
19	record	0	2	0	0	1	0
20	repeat	0	2	0	0	1	0
21	integer	0	2	0	0	1	0
22	real	0	2	0	0	1	0
23	boolean	0	2	0	0	1	0
24	char	0	2	0	0	1	0
25	string	0	2	0	0	1	0
26	then	0	2	0	0	1	0
27	to	0	2	0	0	1	0
28	type	0	2	0	0	1	0
29	until	0	2	0	0	1	0
30	var	0	2	0	0	1	0
31	while	0	2	0	0	1	0
32	(reserved)	0	2	0	0	1	0
33	integer	0	2	1	0	1	0
34	real	33	2	2	0	1	0
35	char	34	2	3	0	1	0
36	boolean	35	2	4	0	1	0
37	string	36	2	5	0	1	0
38	true	37	0	4	0	1	1

39	false	38	0	4	0	1	0	0
40	writeln	39	3	0	0	1	0	0
41	readln	40	3	0	0	1	0	0
42	arithmetic	41	5	0	0	1	0	0
43	a	42	1	1	0	1	0	0
44	b	43	1	1	0	1	0	0
45	x	44	1	2	0	1	0	0
46	y	45	1	2	0	1	0	0
47	res	46	1	2	0	1	0	0

===== BTAB =====

IDX	LAST	LPAR	PSZE	VSZE
0	47	0	0	5

===== ATAB =====

IDX	XTYP	ETYP	EREF	LOW	HIGH	ELSZ	SIZE
-----	------	------	------	-----	------	------	------

3. Kasus 3

Input:

```

program ControlFlow;
var
  i: integer;
  found: boolean;
begin
  i := 1;
  found := false;
  while i <= 10 do
  begin
    if i mod 2 = 0 then
    begin
      found := true;
      writeln(i)
    end;
    i := i + 1
  end;
  repeat
    i := i - 1
  until i = 0
end.

```

Output

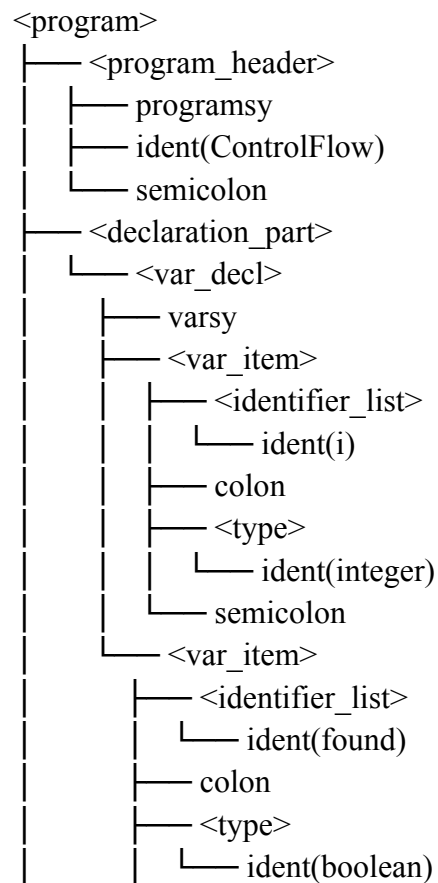
```
programsy
ident (ControlFlow)
semicolon
varsy
ident (i)
colon
ident (integer)
semicolon
ident (found)
colon
ident (boolean)
semicolon
beginsy
ident (i)
becomes
intcon (1)
semicolon
ident (found)
becomes
ident (false)
semicolon
whilesy
ident (i)
leq
intcon (10)
dosy
beginsy
ifsy
ident (i)
imod
intcon (2)
eq
intcon (0)
thensy
beginsy
ident (found)
becomes
ident (true)
semicolon
ident (writeln)
lparent
ident (i)
rparent
```

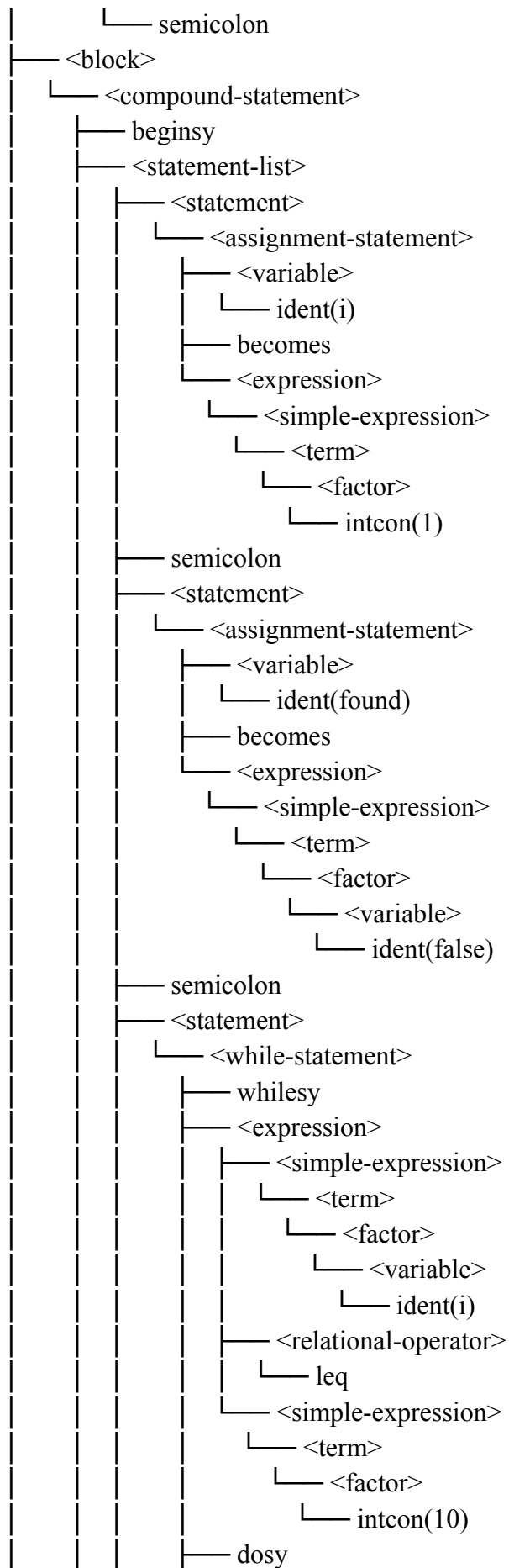
```

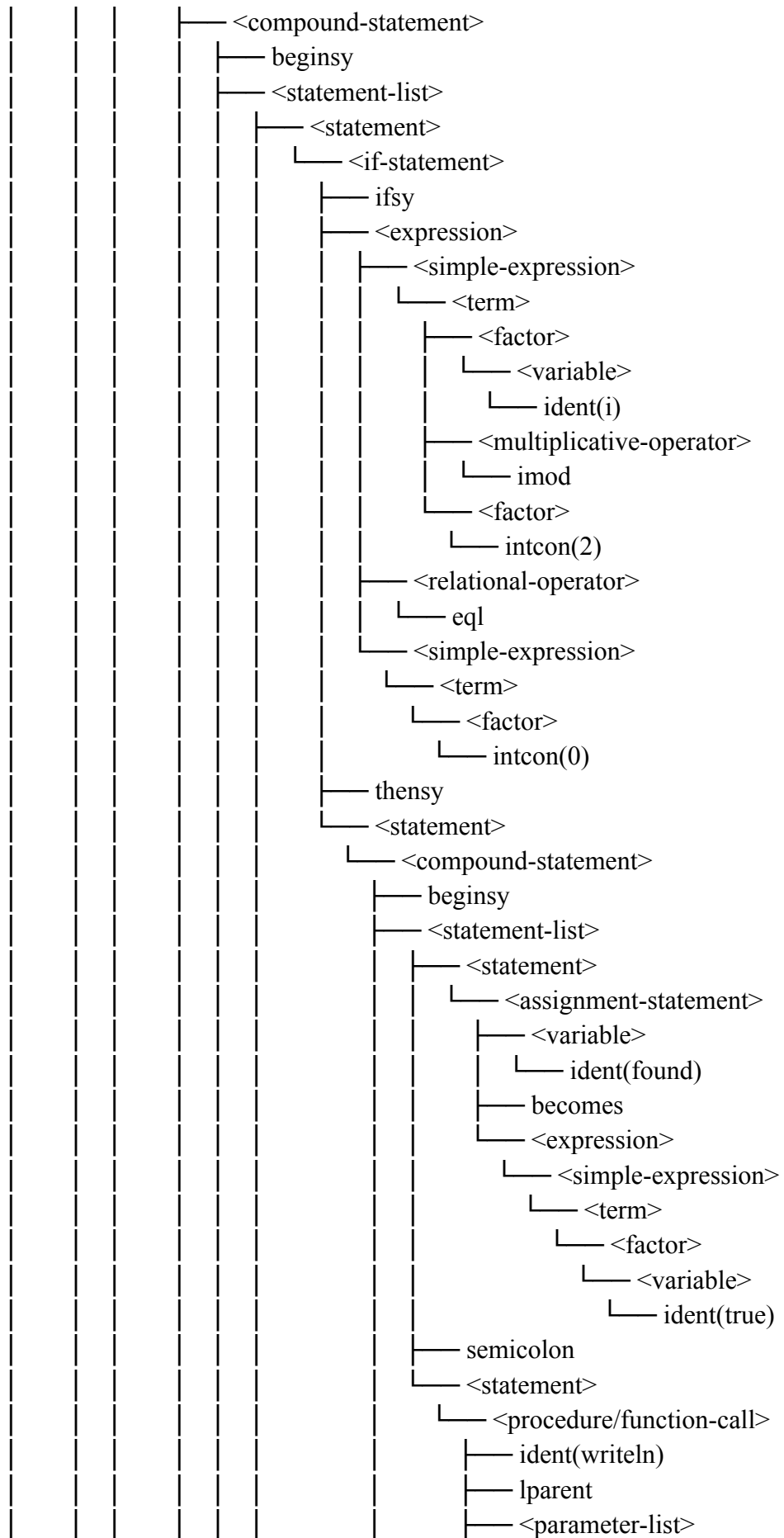
endsy
semicolon
ident (i)
becomes
ident (i)
plus
intcon (1)
endsy
semicolon
repeatsy
ident (i)
becomes
ident (i)
minus
intcon (1)
untilsy
ident (i)
eql
intcon (0)
endsy
period

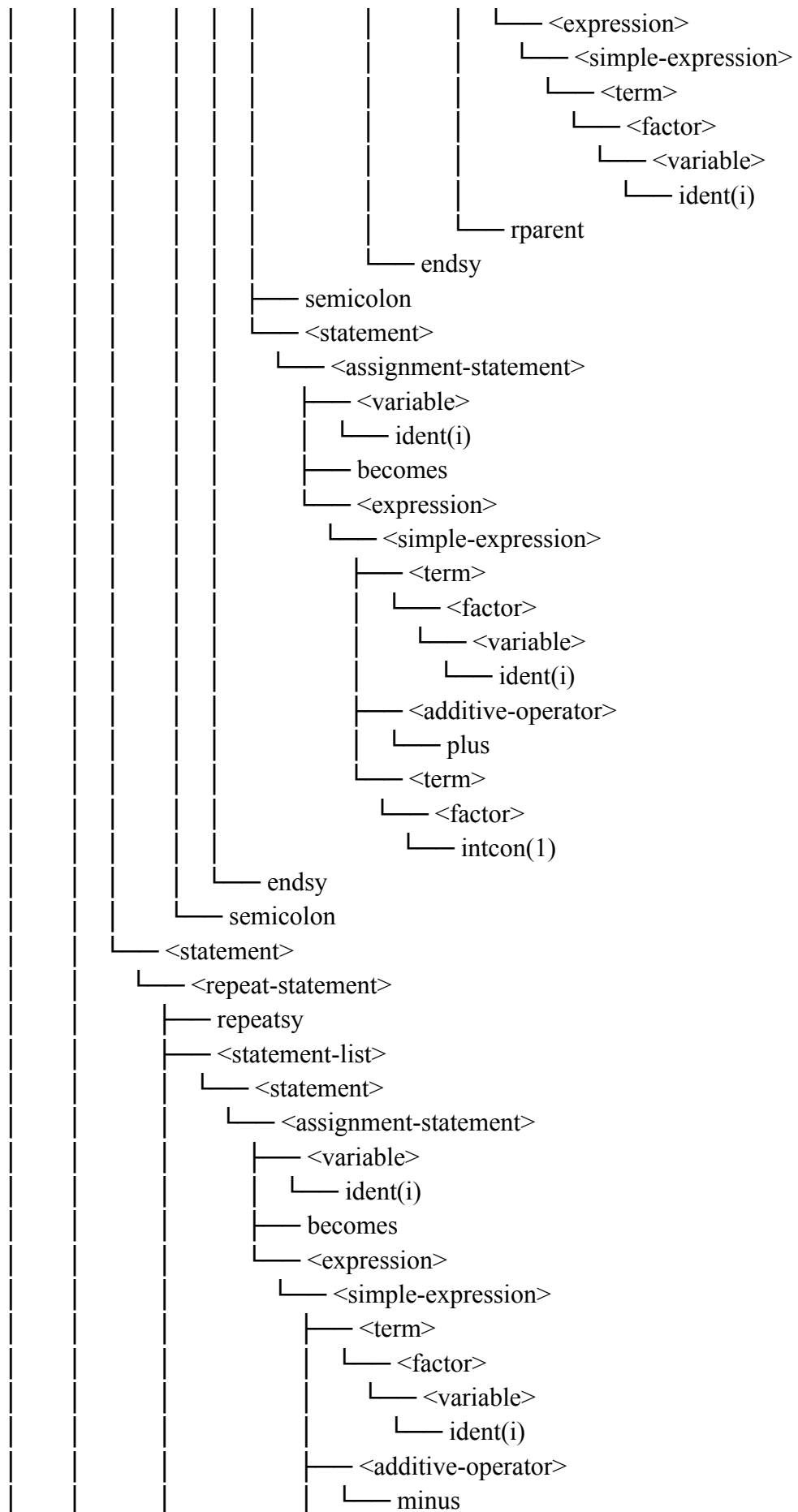
```

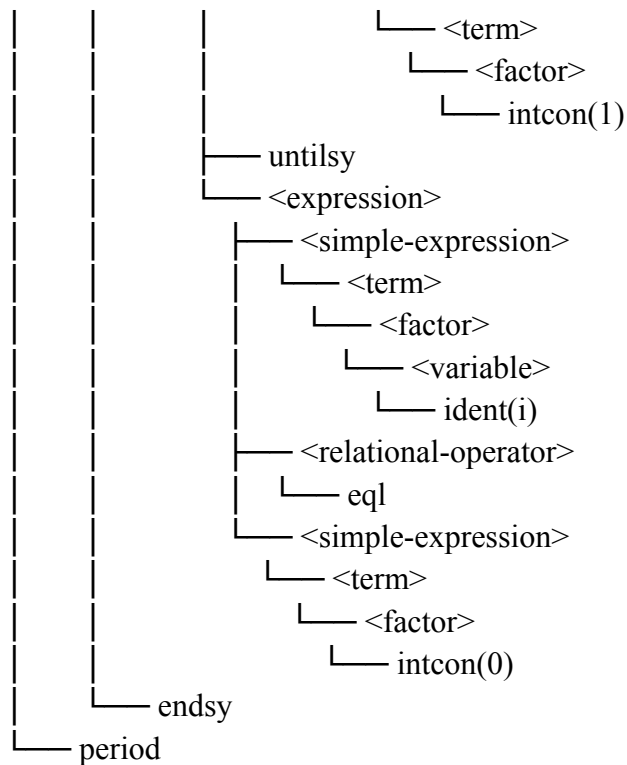
--- PARSE TREE ---











--- ABSTRACT SYNTAX TREE ---

ProgramNode(name: 'ControlFlow')

Declarations

VarDecl(name: 'i')

TypeName(integer)

VarDecl(name: 'found')

TypeName(boolean)

Block

BlockNode

Assign

target:

Var('i')

value:

IntegerNumber(1)

Assign

target:

Var('found')

value:

Var('false')

While

condition:

BinOp('<=')')

Var('i')

IntegerNumber(10)

body:

```

BlockNode
  If
    condition:
      BinOp('=')
      BinOp('mod')
      Var('i')
      IntegerNumber(2)
      IntegerNumber(0)
    then:
      BlockNode
        Assign
          target:
            Var('found')
          value:
            Var('true')
            ProcedureCall(name: 'writeln')
            Var('i')
      Assign
        target:
          Var('i')
        value:
          BinOp('+')
          Var('i')
          IntegerNumber(1)
      Repeat
        body:
          BlockNode
            Assign
              target:
                Var('i')
              value:
                BinOp('-')
                Var('i')
                IntegerNumber(1)
            until:
              BinOp('=')
              Var('i')
              IntegerNumber(0)

--- DECORATED AST ---
ProgramNode(name: 'ControlFlow')
  Declarations
    VarDecl(name: 'i')
    TypeName(integer)

```

```

VarDecl(name: 'found')
  TypeName(boolean)
Block
  BlockNode
  Assign
    target:
      Var('i' : integer [tab=43, lev=0])
    value:
      IntegerNumber(1 : integer)
  Assign
    target:
      Var('found' : boolean [tab=44, lev=0])
    value:
      Var('false' : boolean [tab=39, lev=0])
  While
    condition:
      BinOp('<=' : boolean)
      Var('i' : integer [tab=43, lev=0])
      IntegerNumber(10 : integer)
    body:
      BlockNode
      If
        condition:
          BinOp('= ' : boolean)
          BinOp('mod' : integer)
          Var('i' : integer [tab=43, lev=0])
          IntegerNumber(2 : integer)
          IntegerNumber(0 : integer)
        then:
          BlockNode
          Assign
            target:
              Var('found' : boolean [tab=44, lev=0])
            value:
              Var('true' : boolean [tab=38, lev=0])
          ProcedureCall(name: 'writeln')
          Var('i' : integer [tab=43, lev=0])
      Assign
        target:
          Var('i' : integer [tab=43, lev=0])
        value:
          BinOp('+ ' : integer)
          Var('i' : integer [tab=43, lev=0])
          IntegerNumber(1 : integer)

```

```

Repeat
  body:
    BlockNode
    Assign
    target:
      Var('i' : integer [tab=43, lev=0])
    value:
      BinOp('-', integer)
      Var('i' : integer [tab=43, lev=0])
      IntegerNumber(1 : integer)
  until:
    BinOp('=', boolean)
    Var('i' : integer [tab=43, lev=0])
    IntegerNumber(0 : integer)

```

===== TAB =====

ADR	IDX	IDENT	LINK	OBJ	TYPE	REF	NRM	LEV
	0	and	0	2	0	0	1	0
	1	array	0	2	0	0	1	0
	2	begin	0	2	0	0	1	0
	3	case	0	2	0	0	1	0
	4	const	0	2	0	0	1	0
	5	div	0	2	0	0	1	0
	6	downto	0	2	0	0	1	0
	7	do	0	2	0	0	1	0
	8	else	0	2	0	0	1	0
	9	end	0	2	0	0	1	0
	10	for	0	2	0	0	1	0
	11	function	0	2	0	0	1	0
	12	if	0	2	0	0	1	0
	13	mod	0	2	0	0	1	0
	14	not	0	2	0	0	1	0
	15	of	0	2	0	0	1	0
	16	or	0	2	0	0	1	0
	17	procedure	0	2	0	0	1	0
	18	program	0	2	0	0	1	0
	19	record	0	2	0	0	1	0
	20	repeat	0	2	0	0	1	0
	21	integer	0	2	0	0	1	0
	22	real	0	2	0	0	1	0
	23	boolean	0	2	0	0	1	0
	24	char	0	2	0	0	1	0
	25	string	0	2	0	0	1	0

26	then	0	2	0	0	1	0	0
27	to	0	2	0	0	1	0	0
28	type	0	2	0	0	1	0	0
29	until	0	2	0	0	1	0	0
30	var	0	2	0	0	1	0	0
31	while	0	2	0	0	1	0	0
32	(reserved)	0	2	0	0	1	0	0
33	integer	0	2	1	0	1	0	0
34	real	33	2	2	0	1	0	0
35	char	34	2	3	0	1	0	0
36	boolean	35	2	4	0	1	0	0
37	string	36	2	5	0	1	0	0
38	true	37	0	4	0	1	0	1
39	false	38	0	4	0	1	0	0
40	writeln	39	3	0	0	1	0	0
41	readln	40	3	0	0	1	0	0
42	controlflow	41	5	0	0	1	0	0
43	i	42	1	1	0	1	0	0
44	found	43	1	4	0	1	0	0

===== BTAB =====

IDX	LAST	LPAR	PSIZE	VSZ
0	44	0	0	2

===== ATAB =====

IDX	XTYP	ETYP	EREF	LOW	HIGH	ELSZ	SIZE
-----	------	------	------	-----	------	------	------

4. Kasus 4

Input:

```

program Subprograms;
var
    result: integer;

function Add(a, b: integer): integer;
begin
    Add := a + b
end;

procedure PrintResult(x: integer);
begin
    writeln(x)
end;

```

```
begin
  result := Add(3, 4);
  PrintResult(result);
  writeln(result)
end.
```

Output

```
programsy
ident (Subprograms)
semicolon
varsy
ident (result)
colon
ident (integer)
semicolon
```

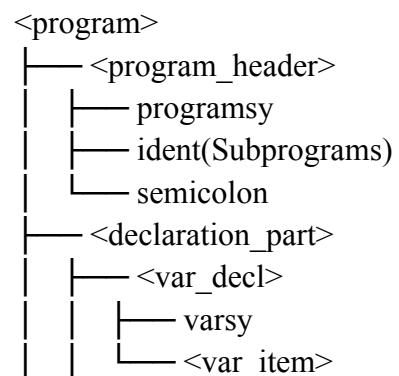
```
functionsy
ident (Add)
lparent
ident (a)
comma
ident (b)
colon
ident (integer)
rparent
colon
ident (integer)
semicolon
beginsy
ident (Add)
becomes
ident (a)
plus
ident (b)
endsy
semicolon
```

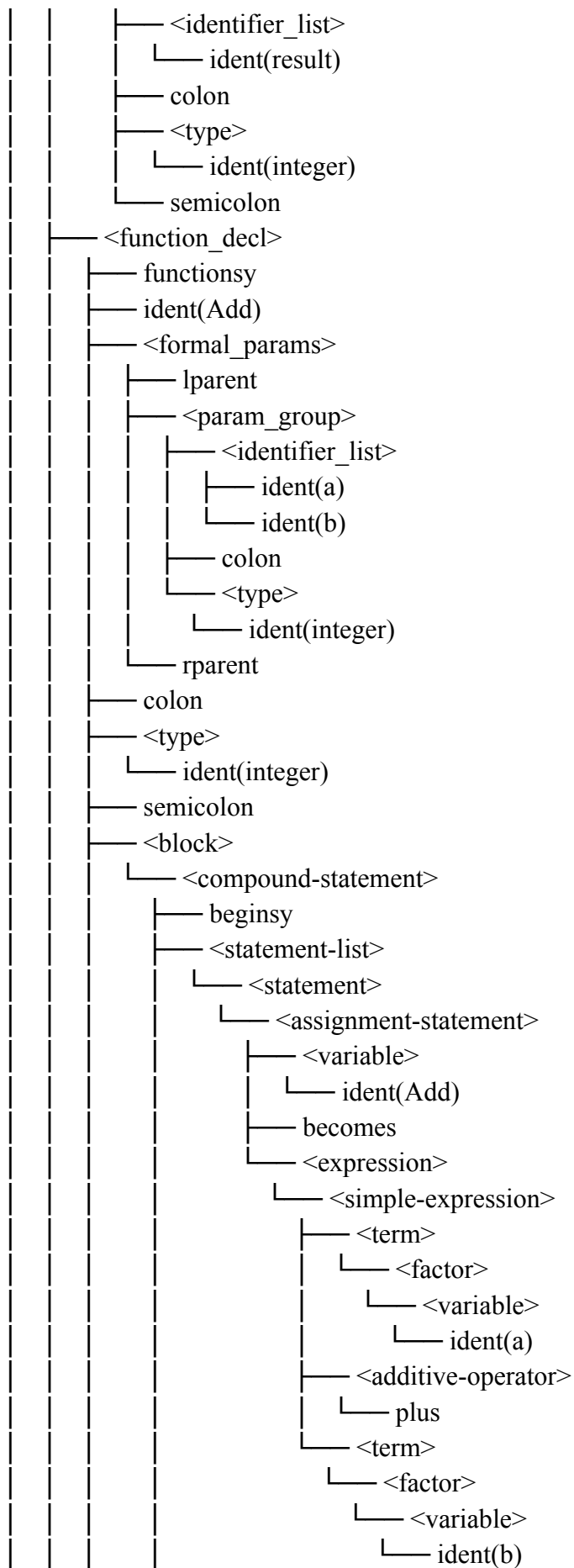
```
proceduresy
ident (PrintResult)
lparent
ident (x)
```

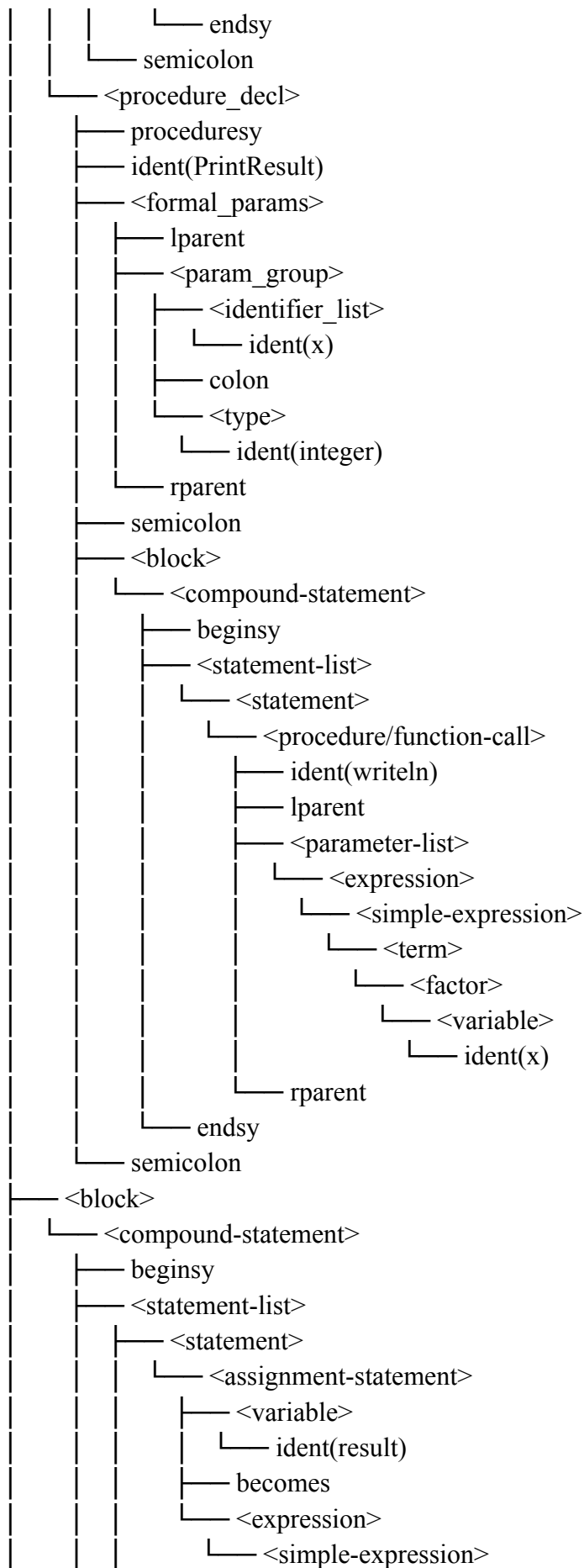

colon
ident (integer)
rparent
semicolon
beginsy
ident (writeln)
lparent
ident (x)
rparent
endsy
semicolon

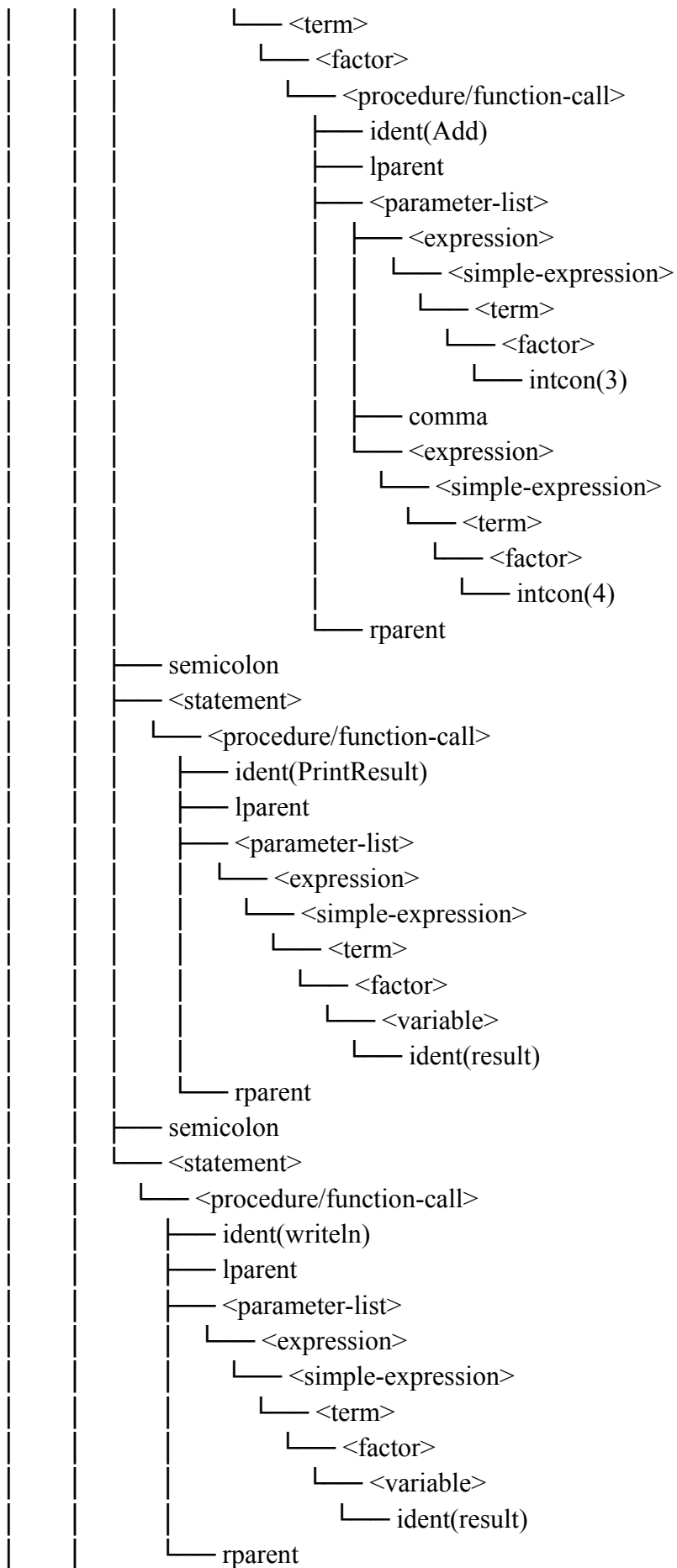
beginsy
ident (result)
becomes
ident (Add)
lparent
intcon (3)
comma
intcon (4)
rparent
semicolon
ident (PrintResult)
lparent
ident (result)
rparent
semicolon
ident (writeln)
lparent
ident (result)
rparent
endsy
period

--- PARSE TREE ---









```

|   └── endsy
└── period

```

--- ABSTRACT SYNTAX TREE ---

ProgramNode(name: 'Subprograms')

Declarations

VarDecl(name: 'result')

TypeName(integer)

FunctionDecl(name: 'Add')

Params

ParamDecl(name: 'a', mode: 'value')

TypeName(integer)

ParamDecl(name: 'b', mode: 'value')

TypeName(integer)

ReturnType

TypeName(integer)

Declarations

Block

BlockNode

Assign

target:

Var('Add')

value:

BinOp('+')

Var('a')

Var('b')

ProcedureDecl(name: 'PrintResult')

Params

ParamDecl(name: 'x', mode: 'value')

TypeName(integer)

Declarations

Block

BlockNode

ProcedureCall(name: 'writeln')

Var('x')

Block

BlockNode

Assign

target:

Var('result')

value:

ProcedureCall(name: 'Add')

IntegerNumber(3)

IntegerNumber(4)

```
ProcedureCall(name: 'PrintResult')
  Var('result')
ProcedureCall(name: 'writeln')
  Var('result')
```

--- DECORATED AST ---

```
ProgramNode(name: 'Subprograms')
  Declarations
    VarDecl(name: 'result')
      TypeName(integer)
    FunctionDecl(name: 'Add')
      Params
        ParamDecl(name: 'a', mode: 'value')
          TypeName(integer)
        ParamDecl(name: 'b', mode: 'value')
          TypeName(integer)
      ReturnType
        TypeName(integer)
      Declarations
      Block
        BlockNode
          Assign
            target:
              Var('Add' : integer [tab=44, lev=0])
            value:
              BinOp('+ ' : integer)
                Var('a' : integer [tab=45, lev=1])
                Var('b' : integer [tab=46, lev=1])
      ProcedureDecl(name: 'PrintResult')
        Params
          ParamDecl(name: 'x', mode: 'value')
            TypeName(integer)
        Declarations
        Block
          BlockNode
            ProcedureCall(name: 'writeln')
              Var('x' : integer [tab=48, lev=1])
      Block
        BlockNode
          Assign
            target:
              Var('result' : integer [tab=43, lev=0])
            value:
              ProcedureCall(name: 'Add')
```

```

IntegerNumber(3 : integer)
IntegerNumber(4 : integer)
ProcedureCall(name: 'PrintResult')
Var('result' : integer [tab=43, lev=0])
ProcedureCall(name: 'writeln')
Var('result' : integer [tab=43, lev=0])

```

===== TAB =====

IDX	IDENT	LINK	OBJ	TYPE	REF	NRM	LEV
ADR							
0	and	0	2	0	0	1	0
1	array	0	2	0	0	1	0
2	begin	0	2	0	0	1	0
3	case	0	2	0	0	1	0
4	const	0	2	0	0	1	0
5	div	0	2	0	0	1	0
6	downto	0	2	0	0	1	0
7	do	0	2	0	0	1	0
8	else	0	2	0	0	1	0
9	end	0	2	0	0	1	0
10	for	0	2	0	0	1	0
11	function	0	2	0	0	1	0
12	if	0	2	0	0	1	0
13	mod	0	2	0	0	1	0
14	not	0	2	0	0	1	0
15	of	0	2	0	0	1	0
16	or	0	2	0	0	1	0
17	procedure	0	2	0	0	1	0
18	program	0	2	0	0	1	0
19	record	0	2	0	0	1	0
20	repeat	0	2	0	0	1	0
21	integer	0	2	0	0	1	0
22	real	0	2	0	0	1	0
23	boolean	0	2	0	0	1	0
24	char	0	2	0	0	1	0
25	string	0	2	0	0	1	0
26	then	0	2	0	0	1	0
27	to	0	2	0	0	1	0
28	type	0	2	0	0	1	0
29	until	0	2	0	0	1	0
30	var	0	2	0	0	1	0
31	while	0	2	0	0	1	0
32	(reserved)	0	2	0	0	1	0
33	integer	0	2	1	0	1	0

34	real	33	2	2	0	1	0	0
35	char	34	2	3	0	1	0	0
36	boolean	35	2	4	0	1	0	0
37	string	36	2	5	0	1	0	0
38	true	37	0	4	0	1	0	1
39	false	38	0	4	0	1	0	0
40	writeln	39	3	0	0	1	0	0
41	readln	40	3	0	0	1	0	0
42	subprograms	41	5	0	0	1	0	0
43	result	42	1	1	0	1	0	0
44	add	43	4	1	0	1	0	0
45	a	0	1	1	0	1	1	0
46	b	45	1	1	0	1	1	0
47	printresult	44	3	0	0	1	0	0
48	x	0	1	1	0	1	1	0

===== BTAB =====

	IDX	LAST	LPAR	PSZE	VSZE
0	47	0	0	1	
1	46	0	0	2	
2	48	0	0	1	

===== ATAB =====

	IDX	XTYP	ETYP	EREF	LOW	HIGH	ELSZ	SIZE
--	-----	------	------	------	-----	------	------	------

5. Kasus 5

Input:

```

program ForLoop;
var
  i: integer;
  total: integer;
begin
  total := 0;
  for i := 1 to 10 do
  begin
    total := total + i;
    writeln(i)
  end;
  writeln(total)
end.

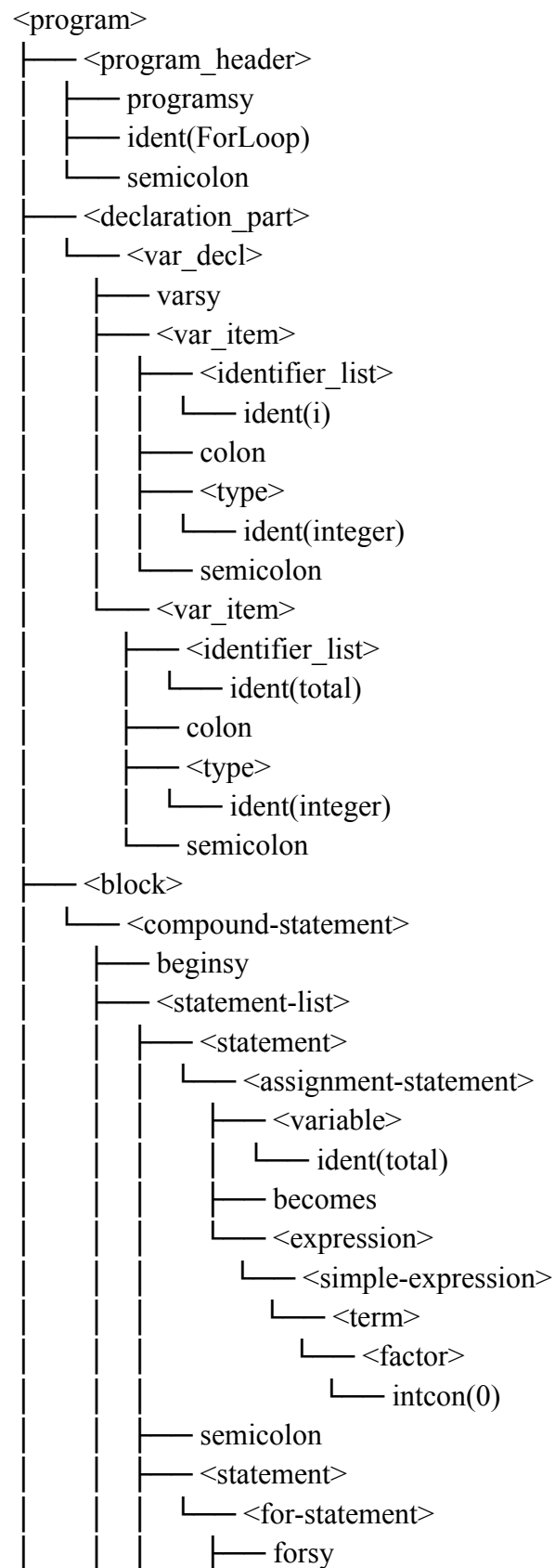
```

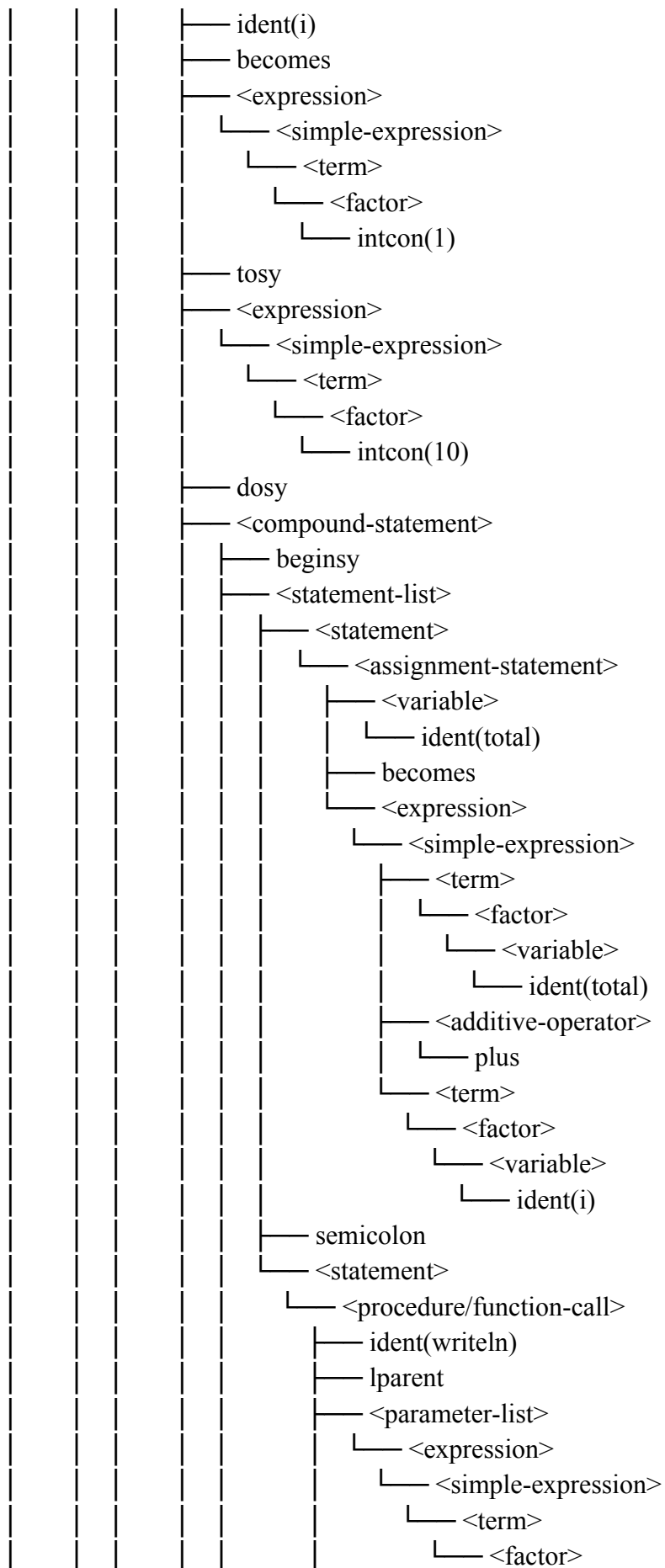

Output

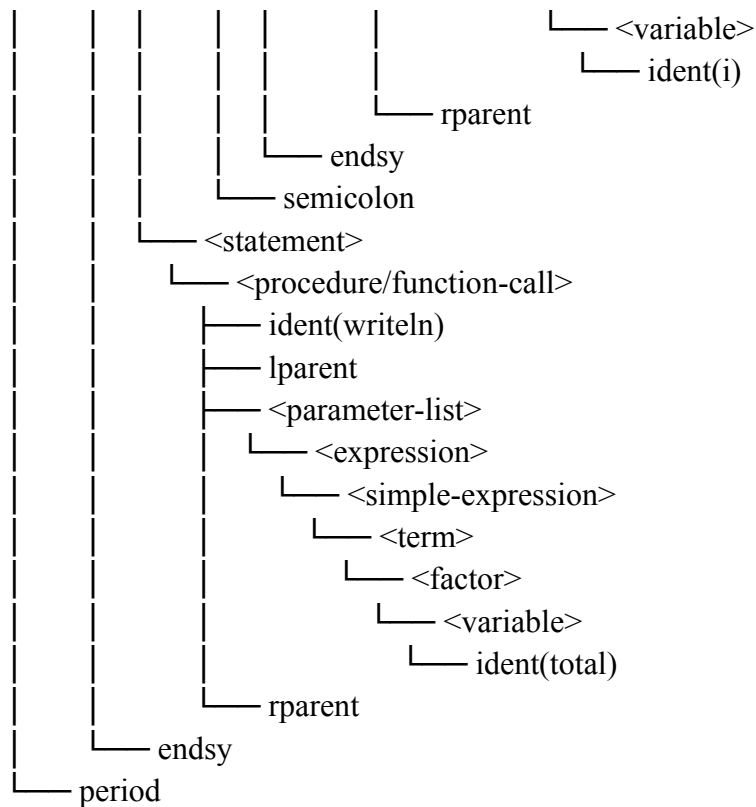
programsy
ident (ForLoop)
semicolon
varsy
ident (i)
colon
ident (integer)
semicolon
ident (total)
colon
ident (integer)
semicolon
beginsy
ident (total)
becomes
intcon (0)
semicolon
forsy
ident (i)
becomes
intcon (1)
tosy
intcon (10)
dosy
beginsy
ident (total)
becomes
ident (total)
plus
ident (i)
semicolon
ident (writeln)
lparent
ident (i)
rparent
endsy
semicolon
ident (writeln)
lparent
ident (total)
rparent

endsy
period

--- PARSE TREE ---







--- ABSTRACT SYNTAX TREE ---

```
ProgramNode(name: 'ForLoop')
```

Declarations

```
VarDecl(name: 'i')
```

TypeName(integer)

```
VarDecl(name: 'total')
```

TypeName(integer)

Block

BlockNode

Assign

target:

Var('total')

value:

IntegerNumber(0)

For(iterator: 'i', direction: 'to')

start:

IntegerNumber(1)

end:

IntegerNumber(10)

body:

BlockNode

Assign

target:

```
Var('total')
```

```

    value:
      BinOp('+')
      Var('total')
      Var('i')
    ProcedureCall(name: 'writeln')
      Var('i')
    ProcedureCall(name: 'writeln')
      Var('total')

```

--- DECORATED AST ---

```

ProgramNode(name: 'ForLoop')
  Declarations
    VarDecl(name: 'i')
      TypeName(integer)
    VarDecl(name: 'total')
      TypeName(integer)
  Block
    BlockNode
      Assign
        target:
          Var('total' : integer [tab=44, lev=0])
        value:
          IntegerNumber(0 : integer)
      For(iterator: 'i', direction: 'to')
        start:
          IntegerNumber(1 : integer)
        end:
          IntegerNumber(10 : integer)
        body:
          BlockNode
            Assign
              target:
                Var('total' : integer [tab=44, lev=0])
              value:
                BinOp('+ : integer)
                Var('total' : integer [tab=44, lev=0])
                Var('i' : integer [tab=43, lev=0])
            ProcedureCall(name: 'writeln')
              Var('i' : integer [tab=43, lev=0])
            ProcedureCall(name: 'writeln')
              Var('total' : integer [tab=44, lev=0])

```

===== TAB =====

IDX	IDENT	LINK	OBJ	TYPE	REF	NRM	LEV
ADR							
0	and	0	2	0	0	1	0
1	array	0	2	0	0	1	0
2	begin	0	2	0	0	1	0
3	case	0	2	0	0	1	0
4	const	0	2	0	0	1	0
5	div	0	2	0	0	1	0
6	downto	0	2	0	0	1	0
7	do	0	2	0	0	1	0
8	else	0	2	0	0	1	0
9	end	0	2	0	0	1	0
10	for	0	2	0	0	1	0
11	function	0	2	0	0	1	0
12	if	0	2	0	0	1	0
13	mod	0	2	0	0	1	0
14	not	0	2	0	0	1	0
15	of	0	2	0	0	1	0
16	or	0	2	0	0	1	0
17	procedure	0	2	0	0	1	0
18	program	0	2	0	0	1	0
19	record	0	2	0	0	1	0
20	repeat	0	2	0	0	1	0
21	integer	0	2	0	0	1	0
22	real	0	2	0	0	1	0
23	boolean	0	2	0	0	1	0
24	char	0	2	0	0	1	0
25	string	0	2	0	0	1	0
26	then	0	2	0	0	1	0
27	to	0	2	0	0	1	0
28	type	0	2	0	0	1	0
29	until	0	2	0	0	1	0
30	var	0	2	0	0	1	0
31	while	0	2	0	0	1	0
32	(reserved)	0	2	0	0	1	0
33	integer	0	2	1	0	1	0
34	real	33	2	2	0	1	0
35	char	34	2	3	0	1	0
36	boolean	35	2	4	0	1	0
37	string	36	2	5	0	1	0
38	true	37	0	4	0	1	0
39	false	38	0	4	0	1	0
40	writeln	39	3	0	0	1	0
41	readln	40	3	0	0	1	0

42	forloop	41	5	0	0	1	0	0
43	i	42	1	1	0	1	0	0
44	total	43	1	1	0	1	0	0

===== BTAB =====

IDX	LAST	LPAR	PSZE	VSZE
0	44	0	0	2

===== ATAB =====

IDX	XTYP	ETYP	EREF	LOW	HIGH	ELSZ	SIZE
-----	------	------	------	-----	------	------	------

6. Kasus 6

Input:

```

program EnumTest;
type
  Color == (red, green, blue);

var
  c: Color;

begin
  c := red;

  if c == green then
  begin
    writeln('green');
  end;
end.

```

Output

```

programsy
ident (EnumTest)
semicolon
typesy
ident (Color)
eql
lparent
ident (red)
comma
ident (green)

```

comma
ident (blue)
rparent
semicolon

varsy
ident (c)
colon
ident (Color)
semicolon

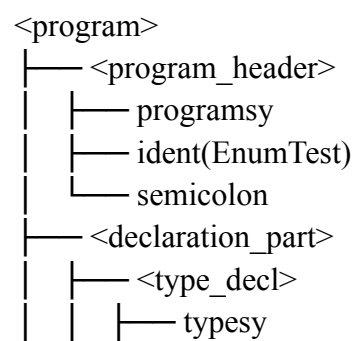
beginsy
ident (c)
becomes
ident (red)
semicolon

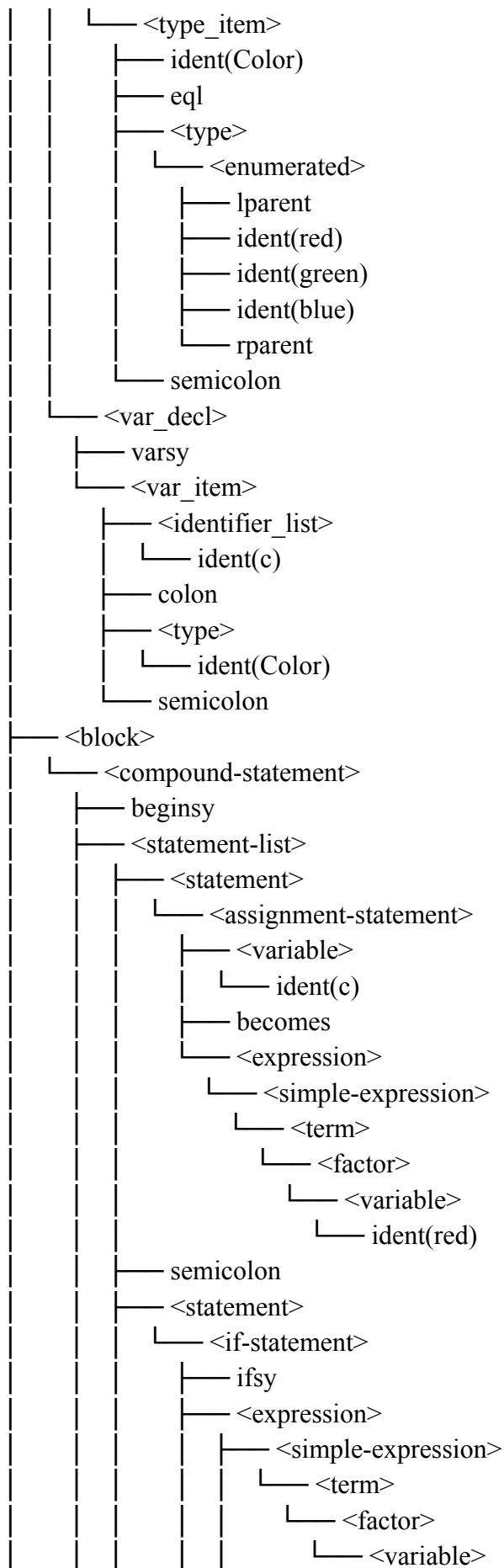
ifsy
ident (c)
eq
ident (green)
thensy
beginsy
ident (writeln)
lparent
string ('green')
rparent
semicolon

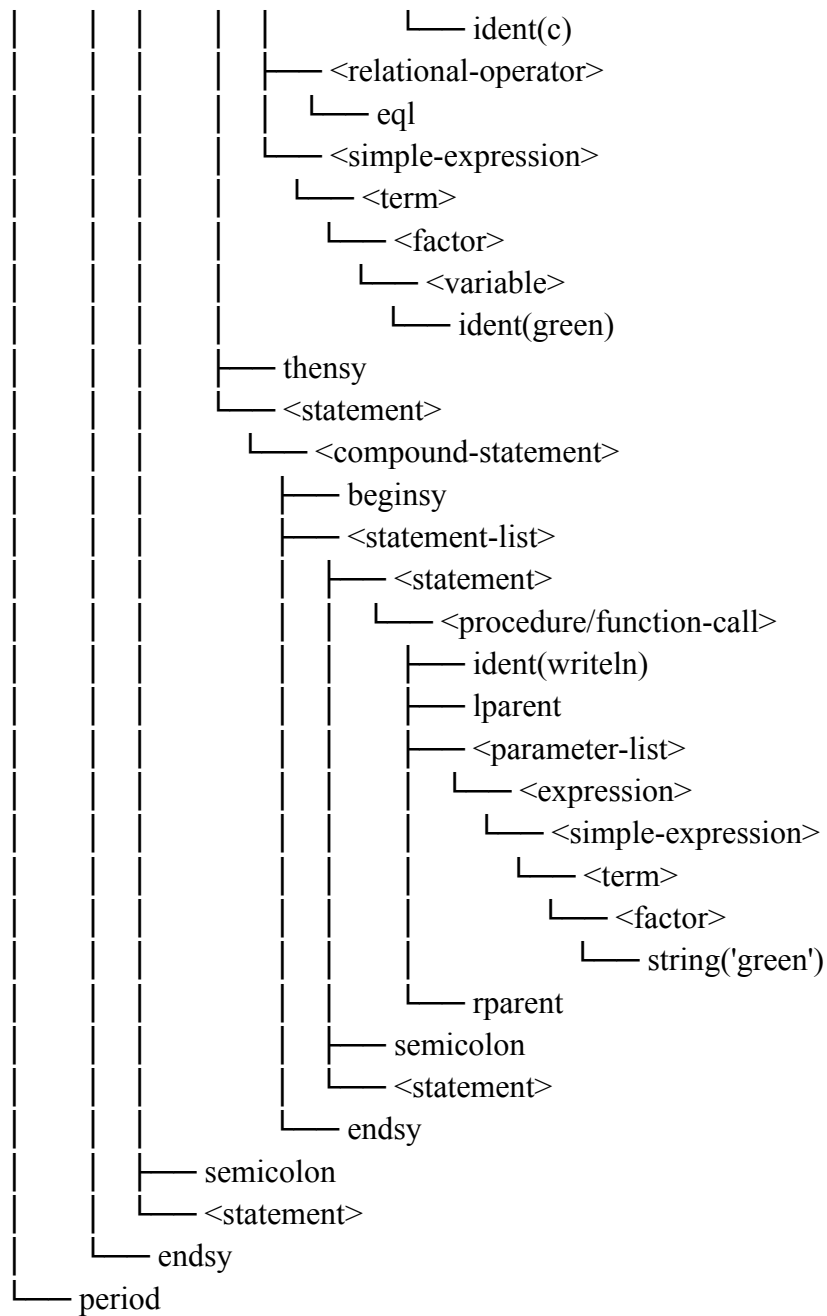
endsy
semicolon

endsy
period

--- PARSE TREE ---







--- ABSTRACT SYNTAX TREE ---

ProgramNode(name: 'EnumTest')

Declarations

TypeDecl(name: 'Color')

EnumType(red, green, blue)

VarDecl(name: 'c')

TypeName(Color)

Block

BlockNode

Assign

target:

Var('c')

```

value:
  Var('red')
If
condition:
  BinOp('==')
  Var('c')
  Var('green')
then:
  BlockNode
  ProcedureCall(name: 'writeln')
  String('green')
  Empty
Empty

```

--- DECORATED AST ---

```

ProgramNode(name: 'EnumTest')
Declarations
  TypeDecl(name: 'Color')
  EnumType(red, green, blue)
  VarDecl(name: 'c')
  TypeName(Color)
Block
  BlockNode
  Assign
  target:
    Var('c' : enum [tab=47, lev=0])
  value:
    Var('red' : enum [tab=44, lev=0])
  If
  condition:
    BinOp('=' : boolean)
    Var('c' : enum [tab=47, lev=0])
    Var('green' : enum [tab=45, lev=0])
  then:
    BlockNode
    ProcedureCall(name: 'writeln')
    String('green' : string)
    Empty
  Empty
Empty

```

===== TAB =====

IDX	IDENT	LINK	OBJ	TYPE	REF	NRM	LEV
ADR							
0	and	0	2	0	0	1	0

1	array	0	2	0	0	1	0	0
2	begin	0	2	0	0	1	0	0
3	case	0	2	0	0	1	0	0
4	const	0	2	0	0	1	0	0
5	div	0	2	0	0	1	0	0
6	downto	0	2	0	0	1	0	0
7	do	0	2	0	0	1	0	0
8	else	0	2	0	0	1	0	0
9	end	0	2	0	0	1	0	0
10	for	0	2	0	0	1	0	0
11	function	0	2	0	0	1	0	0
12	if	0	2	0	0	1	0	0
13	mod	0	2	0	0	1	0	0
14	not	0	2	0	0	1	0	0
15	of	0	2	0	0	1	0	0
16	or	0	2	0	0	1	0	0
17	procedure	0	2	0	0	1	0	0
18	program	0	2	0	0	1	0	0
19	record	0	2	0	0	1	0	0
20	repeat	0	2	0	0	1	0	0
21	integer	0	2	0	0	1	0	0
22	real	0	2	0	0	1	0	0
23	boolean	0	2	0	0	1	0	0
24	char	0	2	0	0	1	0	0
25	string	0	2	0	0	1	0	0
26	then	0	2	0	0	1	0	0
27	to	0	2	0	0	1	0	0
28	type	0	2	0	0	1	0	0
29	until	0	2	0	0	1	0	0
30	var	0	2	0	0	1	0	0
31	while	0	2	0	0	1	0	0
32	(reserved)	0	2	0	0	1	0	0
33	integer	0	2	1	0	1	0	0
34	real	33	2	2	0	1	0	0
35	char	34	2	3	0	1	0	0
36	boolean	35	2	4	0	1	0	0
37	string	36	2	5	0	1	0	0
38	true	37	0	4	0	1	0	1
39	false	38	0	4	0	1	0	0
40	writeln	39	3	0	0	1	0	0
41	readln	40	3	0	0	1	0	0
42	enumtest	41	5	0	0	1	0	0
43	color	42	2	9	0	1	0	0
44	red	43	0	9	0	1	0	0

45	green	44	0	9	0	1	0	1
46	blue	45	0	9	0	1	0	2
47	c	46	1	9	0	1	0	0

===== BTAB =====

IDX	LAST	LPAR	PSZE	VSZE
0	47	0	0	1

===== ATAB =====

IDX	XTYP	ETYP	EREF	LOW	HIGH	ELSZ	SIZE
-----	------	------	------	-----	------	------	------

Bab 4. Kesimpulan dan Saran

Pada Milestone 3 ini, telah berhasil diimplementasikan Semantic Analysis. Implementasi mencakup tiga komponen utama yang saling terhubung: **ASTBuilder**, **SymbolTable**, dan **SemanticAnalyzer**.

Hasil Uji

- Program mampu membangun AST yang ringkas dari parse tree dengan membuang informasi sintaksis yang tidak diperlukan (seperti token BEGIN, END, SEMICOLON, dan kurung redundan), sehingga hanya hubungan semantik antar operator dan operand yang dipertahankan.
- Symbol table berfungsi dengan benar dalam mengelola scope dan menyimpan informasi identifier. Mekanisme pushScope/popScope dan rantai link per blok memungkinkan penelusuran identifier dari scope terdalam ke scope terluar dengan benar.
- Pemeriksaan tipe berjalan dengan benar untuk kasus-kasus dasar: assignment antara tipe inkompatibel terdeteksi dan dilaporkan, ekspresi kondisi pada if/while/repeat divalidasi harus bertipe Boolean, dan aturan aritmetika (integer/real mixing, operator / menghasilkan real) diterapkan secara konsisten.
- Decorated AST terannotasi dengan informasi tipe, indeks symbol table, dan lexical level pada setiap node yang relevan.
- Predefined identifier (integer, real, char, boolean, string, true, false, writeln, readln) telah terdefinisi di symbol table sebelum analisis dimulai, sehingga program Arion dapat menggunakannya tanpa deklarasi eksplisit.
- Selain semantic checking dasar seperti type checking dan scope resolution, compiler juga berhasil mengimplementasikan enumerated type menggunakan TYPE_ENUM beserta semantic checking terhadap assignment dan relational expression antar enum.

Saran

- Penambahan Warning: Selain error yang menghentikan kompilasi, dapat ditambahkan mekanisme warning untuk kasus seperti variabel yang dideklarasikan tetapi tidak pernah digunakan (unused variable) atau kode yang tidak dapat dicapai (unreachable code) setelah return.

Lampiran

Tautan Repository Github: <https://github.com/Beeziebloop/ZZZ-Tubes-IF2224-2026>

Pembagian Tugas:

NIM	Nama	Pembagian Tugas
13524125	Muhammad Rafi Akbar	Type Checking + Dekorasi AST + Error Handling,

		Laporan
13524135	Varistha Devi	AST Node Definitions + Parse Tree → AST Conversion, Laporan
13524138	Ahmad Rinofaros Muchtar	Symbol Table (tab, btab, atab) + Scope Management, Laporan

Referensi

1. <https://dev.to/balapriya/abstract-syntax-tree-ast-explained-in-plain-english-1h38>
2. <https://medium.com/@milhamsuryapratama/abstract-syntax-tree-ast-91440fbca59f>
3. <https://www.geeksforgeeks.org/compiler-design/semantic-analysis-in-compiler-design/>
4. https://www.tutorialspoint.com/compiler_design/compiler_design_semantic_analysis.htm
5. <https://www.geeksforgeeks.org/compiler-design/symbol-table-compiler/>
6. https://www.tutorialspoint.com/compiler_design/compiler_design_symbol_table.htm
7. <https://medium.com/@Saman-Mahmood/symbol-table-in-compiler-construction-13970a0025d8>
8. <https://www.naukri.com/code360/library/symbol-tables-in-compiler-design>
9. <https://www.geeksforgeeks.org/compiler-design/type-checking-in-compiler-design/>
10. <https://www.naukri.com/code360/library/type-checking>
11. <https://www.sciencedirect.com/topics/computer-science/enumerated-type>
12. <https://softwareengineering.stackexchange.com/questions/264931/enumerated-types-and-their-interpretation-by-compilers>